

“Programmer’s Guide”

*Course: Measurements for Automation
and Industrial Production*



Prepared by:

ANTONIO PEDICINI (389000080)
MIRIAM PARADISO (389000052)
HAMZA MEKNI (389000063)
OMID MORADIARSHAD (389000067)



Table of contents

Abstract	6
I Heat Transfer for a Fluid Flowing in a Tube	7
II Programmable Hardware Instrumentation Used	9
Power Supply Agilent E36436A	pg.9
NI DAQ USB-6008 data acquisition	pg.10
Arduino UNO R3	pg.13
III Signal Conditioning Circuit	15
Circuit Design	pg.15
INA128P	pg.16
Potentiometer	pg.17
Circuit Interconnections	pg.17
IV Software	19
LabVIEW Virtual Instrument	pg.19
Finite State Machine (FSM) LabVIEW Design Pattern	pg.20
Producer-Consumer LabVIEW Design Pattern	pg.21
main.vi	pg.22
Producer Loop	pg.24
Arduino & Power Supply Controls Consumer Loop	pg.26
DAQ Consumer Loop	pg.29
daq_fsm.vi	pg.33
Arduino_FlowPump_Controller.vi	pg.35

Arduino_Serial_FSM.vi	pg.35
Agilent_GPIB_FSM.vi	pg.39
Third Consumer:Current Ramp State Machine	pg.40
External Commands Decoder Loop	pg.43
V Calibration	44
Calibration Loop	pg.44
Histograms: Samples Statistical Distribution	pg.45
Results: Expected Value and Standard Deviation	pg.46
Standard and Expanded Uncertainty	pg.47
Results: Datasets Uncertainty	pg.49
Linear Regression	pg.50
Calibration Curve	pg.51
Combined Uncertainty	pg.53
Calibration Report	pg.54
VI Results	55
Calibration Dataset Analysis	pg.55
Ascending and Descending phases curves	pg.55
Ascending and Descending phases: Linear Regression	pg.56
Calibration Curves	pg.57
Measuring System Combined Uncertainty	pg.58
VII Conclusions	60
Project Workflow	pg.60
Concluding Remarks	pg.63
A Arduino Script	64
Circuit	pg.64

List of Figures

1	Heat transfer to a fluid flowing in a tube.	7
2	Structure of thermal mass flowmeter.	8
3	Power Supply E3634A	9
4	DAQ USB-6008 overview.	10
5	DAQ USB-6008 Schematic Block Diagram	11
6	DAQ USB-6008 - LabVIEW.	11
7	DAQ USB-6008 Pins	13
8	Arduino UNO R3 Pins [5]	13
9	INA128P package (a) and its corresponding simplified schematic (b).	16
10	Pin INA128P	16
11	Experimental setup components: Power supplies and the backside of the conditioning circuit board.	18
12	Overview of the measuring system: Physical setup (left) and detailed schematic (right).	18
13	Icons representing different components (VIs) of the main developed software.	19
14	Finite State Machine (FSM) Example Implementation in LabVIEW	21
15	Producer Consumer Design Pattern Example Implementation	22
16	<i>main.vi Producer-Consumer Queued State Machine</i>	23
17	A collection of four event handling cases, demonstrating how the Producer schedules state transitions for the Consumer loops that run in parallel.	24
18	State Diagram of the "Arduino & Power Supply Controls" consumer loop in main.vi.	26
19	A collection of four frames from the sequence of operations that execute in the INIT state within the first consumer loop. The loop initializes the necessary controls to start the subVI Arduino.FlowPump_Controller.vi asynchronously	27

20	State Diagram of the "DAQ Controls" consumer loop in the main.vi. The DAQ board connection is managed by the <i>daq_fsm.vi</i> subVI.	30
21	A collection of the frames that executes in sequence when the DAQ consumer loop enters the READ state. This happens whenever the user requests to acquire new samples during a calibration procedure run, by pressing the READ button on the front panel. .	31
22	Acquisition Tab in main.vi shows the Control panel and the many interactive controls such as the <i>READ</i> button, the <i>Next button</i> and the <i>Samples control</i>	32
23	histogram.vi subVI used to create all the histograms for each feature column in a given dataset	33
24	A collection of the frames that executes in sequence when the DAQ consumer loop enters the READ state. This happens whenever the user requests to acquire new samples during a calibration procedure run, by pressing the READ button on the front panel. .	34
25	<i>Serial Connection FSM</i> consumer loop in <i>Arduino_FlowPump_Controller.vi</i> subVI.	36
26	<i>Arduino_Serial_FSM.vi</i> Block Diagram.	36
27	Finite State Machine State Diagram of " <i>Arduino_Serial_FSM.vi</i> " subVI started asynchronously in <i>Arduino_FlowPump_Controller.vi</i> subVI	37
28	State and Args subVI used to decode the next state transition in the <i>Arduino_Serial_FSM.vi</i>	38
29	Asynchronous Read state used to read measurements asynchronously from Arduino board	39
30	<i>Agilent_GPIB_FSM.vi</i> Block Diagram.	40
31	Current Ramp Flow diagram	41
32	Current Ramp FSM in the <i>Arduino_FlowPump_Control</i> subVI.	42
33	(a) statistics.lvlib subVIs. (b) CALIBRATION.lvlib subVIs.	44
34	Calibration Procedure Loop: histogram and boxplots graphs and statistical metrics computation.	45
35	histogram.vi subVI used to create all the histograms for each feature column in a given dataset	46
36	create_plot.vi subVI used to create all the boxplots for each feature column in a given dataset	46

37	whisky.vi subVI used to compute the lower and upper whiskers for each feature column in a given dataset	46
38	mediano.vi subVI used to compute the lower and upper quartiles and the median value for each feature column in a given dataset	46
39	Calibration Procedure Loop: expected values and standard deviations evaluation phase.	47
40	mean_and_deviation.vi subVI used to compute <i>expected values</i> values and <i>standard devi-</i> <i>ations</i> in feature columns in each dataset	47
41	Calibration Procedure Loop: Hysteresis evaluation phase.	49
42	uncertainty_datasets.vi subVI used to compute the Type A standard uncertainty for each dataset feature.	50
43	pbp_uncertainty.vi subVI used in the uncertainty_datasets.vi to compute the point-by- point standard uncertainty	50
44	Calibration Procedure Loop: curve fitting phase.	51
45	Curve Fitting subVIs used to compute the least squares linear regression estimators and the relative goodness parameters.	51
46	Calibration Procedure Loop: calibration curve linear regression.	52
47	Calibration Procedure Loop: measuring system combined uncertainty.	53
48	FS-1012 (a) ascending and descending curves, with statistical parameters during (b) ascending phase (c) descending phase	55
49	Ascending and Descending phases output when the temperature of the flowing gas is not held constant during the calibration cycle.	56
50	Best-Fit Estimators: FS-1012 (a), FS-2012 (b)	57
51	Calibration Results	57
52	Uncertainty Budget, with all the relevant sources of uncertainty in the measuring system.	58
53	Overview of the Arduino UNO R3 - Digital Transducer Connection.	64

Abstract

This document provides the necessary background information on the calibration methods and the software applications used, with the ultimate goal of producing a guide for the programmers who wish to either modify or extend the capabilities of the software suite developed. This guide is an attempt at making life easier for the user by describing how each step in the calibration procedure has been implemented and how each device connection has been handled. The project's aim is to develop an automated measurement system for characterizing the Renesas FS-1012-1100-NG transducer. This thermal mass flow transducer's output is a voltage signal. This signal has to be analysed and compared with the measurement produced by a pre-calibrated transducer of similar characteristics. For this aim, the Renesas FS-2012-1100-NG was chosen. The task involves several steps, which are all dealt with using a unique software application developed within the LabVIEW development environment: the main VI. Nevertheless, this program calls several other sub-routines to leverage multithreading and parallel programming to handle multiple device connections simultaneously, and to manage each user request in response to a GUI event. Finally, the project's main software application includes a set of data analysis tools, which bring up the statistical distribution of samples in each data set acquired, reveal the presence of Hysteresis, and compute the combined standard uncertainty resulting from a set of uncertainty sources within the measuring system itself.

Heat Transfer for a Fluid Flowing in a Tube

The thermal mass flow sensor used in this work is the calorimetric type, where sensing elements located at opposite ends of a heater detect different temperatures due to air flow.

At the basis of the calorimetric sensing principle, for a fluid (gas or liquid) flowing in a tube, is the energy balance: $\dot{Q} = \dot{m}c_p(T_e - T_i)$ [7], where $\dot{Q}$ is the rate of heat transfer (watts, W) from the heater located in the sensor module to the fluid in motion, $\dot{m}$ is the mass flow rate, c_p is the specific heat of the fluid, and T_i , T_e are the temperatures before and after heat is transferred to the fluid, as illustrated in Figure 1.

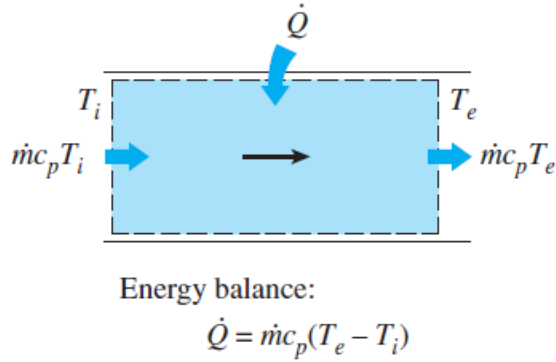


Figure 1: Heat transfer to a fluid flowing in a tube.

There are several methods for measuring the heat transfer in thermal mass flow sensors, including thermoresistivity and thermoelectrics. Thermoelectric flow sensors use thermopiles to measure temperature transfer as a medium flows by. Thermopiles are essentially a set of individual thermocouples that are connected together in series, to increase the sensor output voltage by summing the voltage from each thermocouple.

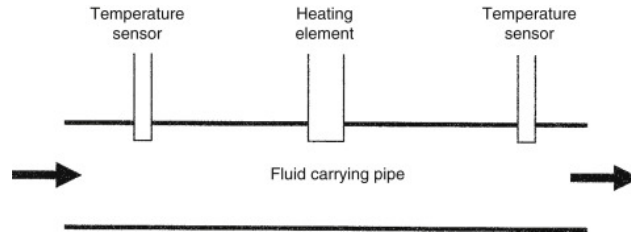


Figure 2: Structure of thermal mass flowmeter.

Thermocouples and, by extension, thermopiles use the Seebeck effect to generate an analog sensing signal. The Seebeck effect explains the phenomenon that a temperature difference between two dissimilar electrical conductors or semiconductors produces a voltage (without the flow of an electrical current). It enables higher sensitivity and unbiased output voltage with no offset or drift.

In MEMS thermal mass flowmeters, of course, the heater and temperature sensors are both micro-electrical mechanical systems (MEMS) scale and contained within a tiny module. These sensors have the advantage of having a fast response time, high accuracy, low power consumption, and high reliability due to the absence of moving parts [9].

Both the Device Under Test (DUT) and the reference instrument used in this project, are of MEMS thermal mass flow sensors. The output of the DUT consists of an analog signal that we compute as the difference between the voltage generated by the two thermocouples, related to the temperature distribution due to fluid motion. The output of the reference instrument, instead, being pre-calibrated, is a digital value which is directly converted into a measure of the mass flow rate, according to the formula reported in the device datasheet.

Our aim is to associate to the DUT output readings an indication of the measured quantity, the mass flow rate, producing a value that is comparable with that of the reference sensor.

Nevertheless, the results produced are based on some assumptions that should always be kept in consideration. In particular, we assume that the fluid in the tube is *laminar*; the *temperature of the fluid is constant* and not affected by external sources, such as the air-pump; *thermophysical properties of the process fluid, such as the specific heat, are constant*. **If these assumptions do not hold**, then, it is necessary to modify the uncertainty evaluation procedure and the formula that computes the measurement system combined uncertainty used in the Results chapter, to account for these model inaccuracies.

Programmable Hardware Instrumentation Used

Power Supply Agilent E36436A

The Agilent E3633A/E3634A DC Power Supply [4] was configured for Constant Voltage (CV) operation, with the output voltage fixed at 12V.

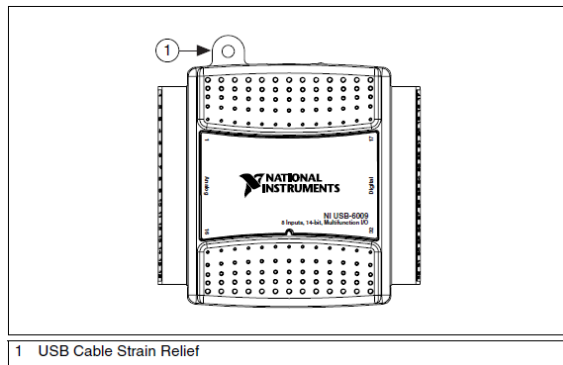


Figure 3: Power Supply E3634A

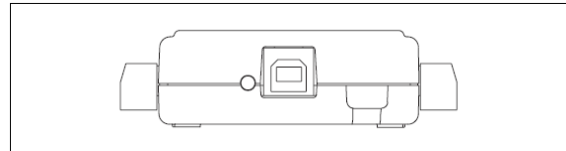
Main characteristics

- **Voltage:** $\pm(\% \text{ of setting} + \text{offset})$
 - **Low Range (0 to 8 V):** $\pm(0.05\% + 5 \text{ mV})$
 - **High Range (0 to 20 V):** $\pm(0.05\% + 15 \text{ mV})$
- **Current:** $\pm(\% \text{ of setting} + \text{offset})$
 - **Low Range (0 to 20 A):** $\pm(0.2\% + 5 \text{ mA})$
 - **High Range (0 to 10 A):** $\pm(0.2\% + 5 \text{ mA})$

NI DAQ USB-6008 data acquisition



(a) USB-6008



(b) USB-6008 back view

Figure 4: DAQ USB-6008 overview.

USB6008 Datasheet¹

The NI USB-6008 [10] provides connection to eight analog input (AI) channels, two analog output (AO) channels, 12 digital input/output (DIO) channels, and a 32-bit counter with a full-speed USB interface.

The following block diagram shows key functional components of the USB-6008:

¹<https://www.ni.com/docs/en-US/bundle/usb-6008-specs/page/specs.html?srsltid=AfmB0oomX5uQUUn1iMT0b3gol5ElTAT813--B6ktlUhKk4vixIqbCJly>

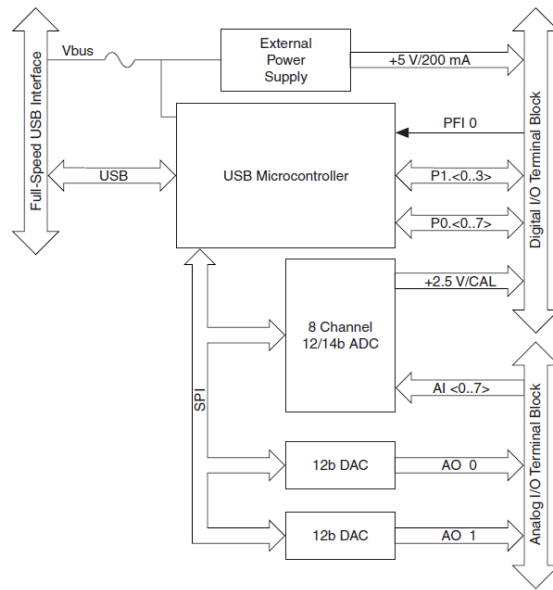


Figure 5: DAQ USB-6008 Schematic Block Diagram

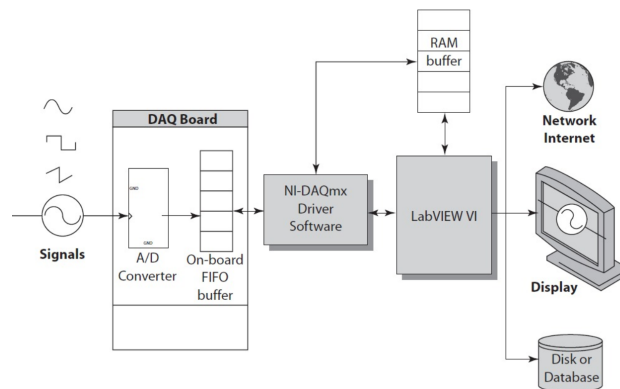


Figure 6: DAQ USB-6008 - LabVIEW.

The capabilities of the NI USB-6008 include:

- **Analog Input:** Acquisition of analog signals from external sensors or devices.
- **Analog Output:** Generation of analog signals for driving actuators or simulating processes.
- **Digital I/O:** Reading and writing of TTL digital signals, crucial for control and logic applications.
- **Counter/Timers:** Counting of digital events using a high-resolution 32-bit counter.

- **Power Sourcing:** Provision of regulated +5V and +2.5V outputs to power external low-power devices.

Internal Architecture

The device incorporates:

- A 12-bit ADC that, through a multiplexer (MUX) and a programmable gain amplifier (PGA), allows to selection and acquisition of one of the analog channels in differential or single-ended mode;
- two 12-bit DACs to generate 0–5 V analog signals;
- an integrated USB microcontroller, which manages data transfer, timing and control logic;
- an internal FIFO for data buffering during acquisition.

All channels are accessible via removable screw terminals, divided between analog and digital terminals, which allow a quick and flexible connection to signal cables (from 16 to 28 AWG).

The device supports:

- Analog acquisition modes:
 - Single-ended (RSE): all channels measure with respect to GND.
 - Differential (DIFF): measurements on channel pairs, with greater noise immunity.
- External digital trigger (via PFI 0) for measurement synchronization with external events.
- Event counter for counting logic transitions (up to 5 MHz).
- Terminal Assignments

Write() and **analogRead()**, along with a Serial Monitor for real-time communication. A typical Arduino Uno board integrates a central **Microcontroller Unit (MCU)** (e.g., ATmega328P) with built-in memory (Flash, SRAM, EEPROM) to execute code and manage I/O. Its diverse pins include **Digital Input/Output (I/O) Pins** (0–13) for digital control, **PWM Pins** (e.g., 3, 5, 6, 9, 10, 11) for analog simulation, and **Analog Input Pins** (A0–A5) connected to a 10-bit Analog-to-Digital Converter (ADC) for reading variable voltages (0–1023). Essential **Power Pins** (5V, 3.3V, GND, VIN) provide regulated power, while a **USB Port** enables PC connection for code upload, power, and serial communication. A **Reset Button** restarts the MCU, and a **Voltage Regulator** ensures stable internal power. In the context of this Project’s work, the Arduino UNO R3 board was used to acquire the digital flow transducer [3] measurements, used as a reference for the calibration procedure. The script that must be installed on the microprocessor is described in the *Appendix A*.

Signal Conditioning Circuit

Circuit Design

The signal conditioning circuit was specifically engineered to amplify the output signals from the analog sensor [2] under calibration. These signals are inherently low-amplitude, rendering them unsuitable for precise detection of airflow variations within the sensor working range. The sensor provides two distinct outputs: a voltage generated by thermopile TP2, exposed to unheated airflow, and a voltage from thermopile TP1, subjected to a temperature variation. The latter exhibits a voltage variation (approximately 15mV at maximum input flow range, as per datasheet specifications) induced by a strategically placed resistor between the two thermopiles. Upon heating, this resistor elevates the temperature of the airflow directed towards the sensor's output, where thermopile TP1 is situated. Consequently, an electrical potential component, attributable to the Seebeck effect, causes a voltage deviation from the no-flow condition.

Nevertheless, experimental measurements using a multimeter corroborate the sensor's datasheet specifications, indicating that the voltages from both thermopiles are in the range of hundreds of millivolts.

Furthermore, to mitigate the risk of overheating the current-controlled air pump, we elected to operate significantly below the sensor's maximum input range, limiting the flow to a maximum of 6 L/min (compared to a specified maximum of 10 L/min). This operational constraint implies that the observed voltage variation will be substantially less than the 15mV indicated in the datasheet.

In essence, the available signals are insufficient to accurately discriminate airflow passage and thus

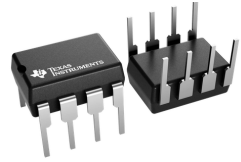
obtain reliable measurements.

Given these limitations, the implementation of a instrumentation amplifier became necessary.

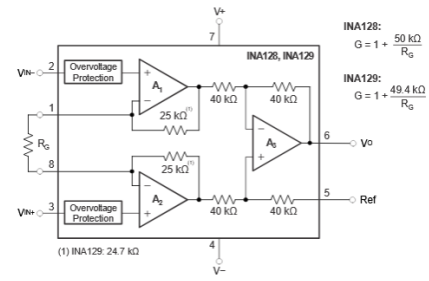
The instrumentation amplifier - INA128P

Fundamentally, the raw signals available are inadequate for reliably discriminating airflow and thus obtaining accurate measurements.

Therefore, the integration of a instrumentation amplifier was deemed essential. We specifically selected the Texas Instruments INA128P instrumentation amplifier.



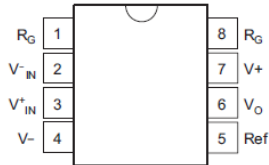
(a)



(b) Simplified Schematic

Figure 9: INA128P package (a) and its corresponding simplified schematic (b).

The INA128 is a low-power, precision instrumentation amplifier, ideally suited for measurement applications demanding high accuracy, minimal drift, and excellent common-mode rejection. Its integrated three-operational amplifier design facilitates a wide range of configurable gains via a single external resistor, thereby ensuring exceptional versatility. Key performance characteristics contributing to the device's efficiency include a low offset voltage (maximum 50 μ V), low quiescent current consumption (700 μ A), and an outstanding Common Mode Rejection Ratio (CMRR) whose minimum values is 120dB.



(a) Pin Configuration

PIN		TYPE	DESCRIPTION
NAME	NO.		
REF	5	Input	Reference input. This pin must be driven by low impedance or connected to ground.
RG	1,8	—	Gain setting pin. For gains greater than 1, place a gain resistor between pin 1 and pin 8.
V-	4	Power	Negative supply
V+	7	Power	Positive supply
V _{IN-}	2	Input	Negative input
V _{IN+}	3	Input	Positive input
V _O	6	Output	Output

(b) Pin Functions

Figure 10: Pin INA128P

Regarding pin configuration, according to the information reported in the component's datasheet in Figure 10, terminals 1 and 8 are designated for connecting the gain resistor; terminal 7 receives the power supply; the output signal is drawn from terminal 6; terminals 4 and 5 are connected to a common ground; and terminals 3 and 2 are connected to the signals from which the difference is to be taken.

The gain factor is calculated using the formula provided in the datasheet, where the amplifier's internal feedback resistance is fixed (50 k Ω), while the gain resistance can be varied by connecting an appropriately valued resistor to terminals 1 and 8. During the experimental phases, the gain resistance was set using a potentiometer to the desired value, according to the gain table in the component's datasheet.

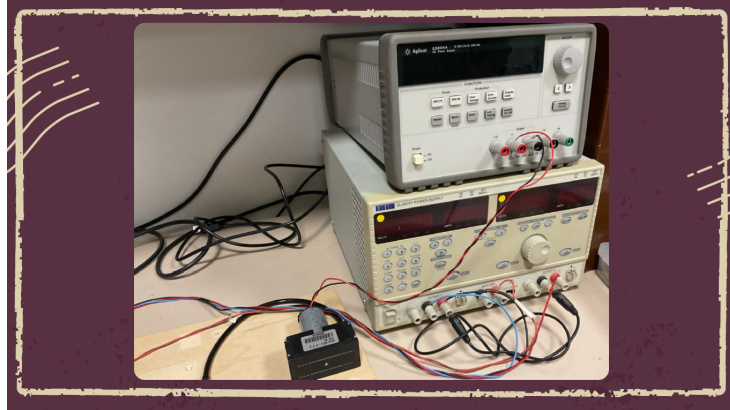
Potentiometer

The designed circuit incorporates a Bourns 3296W-1-103LF multi-turn precision potentiometer, employed for fine-tuning the gain of the instrumentation amplifier. The potentiometer is configured as a resistive divider, with its central terminal (also known as wiper) connected to the amplifier's feedback node. Rotating the trimmer clockwise increases the resistance between the wiper and the CCW terminal, thereby adjusting the gain based on the input signal conditions. Through the potentiometer, the gain resistance was set to 100 Ω , with a tolerance of $\pm 10\%$ as per the device's datasheet. This configuration corresponds to a gain factor of 500. Consequently, if the initial difference between the two signals varies by a few millivolts, the resulting amplified signal will vary between a few hundred millivolts. The potentiometer was connected to the pins 1 and 8 of the used INA128P device.

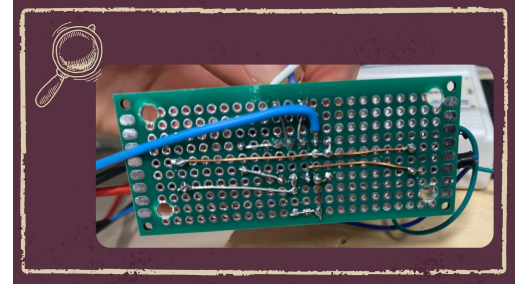
Devices Interconnections

The conditioning circuit is connected to the measurement system instruments via jumper wires. These interconnections are fully explained in the *User's Manual*. Hence, please refer to that document for more information. In this section, simple illustrations are provided to the reader, offering a detailed look at an experimental setup. Figure 11a shows the power supplies, stacked one on top of the other. Next, in Figure 11b, the back of the design Conditioning Circuit is shown. This view reveals the complexity of the soldered connections, which are essential for processing and preparing the signals from the sensors before they can be interpreted or used by the system. Then, Figure 12a presents an

overview of the automated measuring system's hardware setup. Here we see the conditioning board mounted on a wooden base, connected via cables to the other components, showing how the various elements are physically integrated. Finally, Figure 12b illustrates the electrical schematic of the whole measuring system, where the only element missing is the PC or workstation used and its connections.

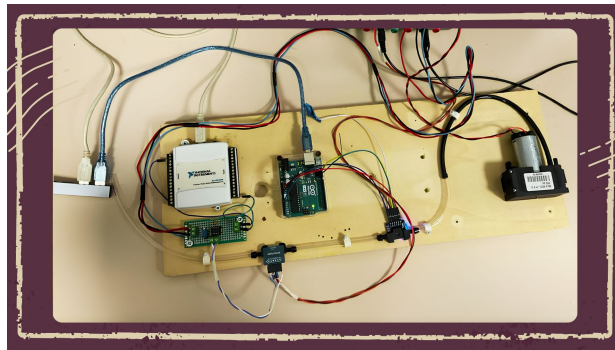


(a) Power Supplies: Agilent E3634A & QL355TP

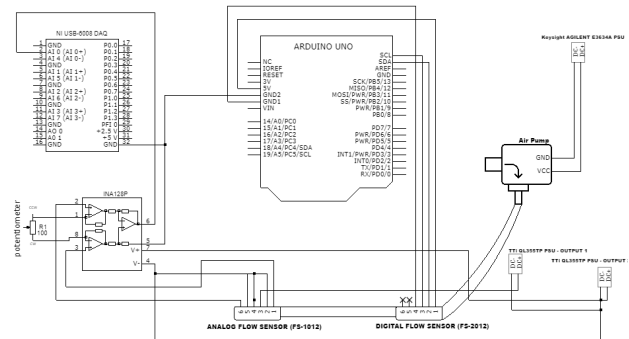


(b) Backside of the Conditioning Circuit Printed Board

Figure 11: Experimental setup components: Power supplies and the backside of the conditioning circuit board.



(a) Physical Measuring System Setup



(b) Schematic of the Whole Measuring System

Figure 12: Overview of the measuring system: Physical setup (left) and detailed schematic (right).

Software



The LabVIEW based Virtual Instrument

The study of the components of the measurement system, along with the delicate task of managing the acquisition of measurement samples simultaneously from multiple devices through different instrument interfaces, led to the choice of the **LabVIEW** development environment. Graphical programming in LabVIEW greatly facilitates the programmer's task, thanks to the wide selection of libraries and software extensions, which offer a high-level abstraction compared to low-level programming of interfaces such as **GPIO** or the **Serial** interface. Furthermore, LabVIEW is a programming environment that was specifically designed to handle the acquisition and generation of input and output signals, respectively, through **DAQ** data acquisition boards.

For these reasons, in order to develop a software system capable of managing the various instruments present in the physical measurement system, we developed several applications (or **Virtual Instruments (VI)**), each tasked with handling communication with a given device through different programming interfaces.



Figure 13: Icons representing different components (VIs) of the main developed software.

Before any modifications can be made to the program, the reader should be familiar with the concepts of *Enum*, *Queue Operations*, *Event Case Structure*, and *Finite State Machines* as implemented in LabVIEW. Therefore, a brief overview of some of the software paradigms used in this project will be provided, with particular focus on Finite State Machines (FSM) and the Producer-Consumer design pattern.

Finite State Machine (FSM) LabVIEW Design Pattern

A **Finite State Machine** (or FSM) is a computational model used to represent a system that can be in only one state at a time, and that transitions from one state to another in response to specific events or conditions. It is one of the most commonly used paradigms for designing control logic, operational sequences, and reactive systems.

In LabVIEW, a finite state machine can be implemented in various ways [6]. For this project, the FSM has been implemented using a queue-based state scheduling mechanism [11], made possible through the **Synchronization/Queue Operations** *sub-palette*. Figure 14 shows the architecture adopted for all FSMs within this project. Each FSM consists of a **while loop** that iteratively executes the machine, a **Guard Clause**—a case structure that handles potential errors during execution—and a second case structure responsible for decoding and executing the current state’s actions, as well as managing the scheduling of the next state.

Outside the while loop of a general Queued-Finite State Machine implementation in LabVIEW, as illustrated in Figure 14, a queue is initialized and used to post state transition commands. The queue’s data type is defined as an **Enum** constant, which maps integer values to string labels, thereby abstracting the machine’s states into human-readable representations.

Among the subVIs implementing this FSM paradigm are **Agilent_GPIB_FSM.vi**, **Arduino_Serial_FSM.vi**, which control the logical sequencing and operations required to remotely manage communication with the **Arduino UNO** board and the **Agilent e3634A** power supply, respectively.

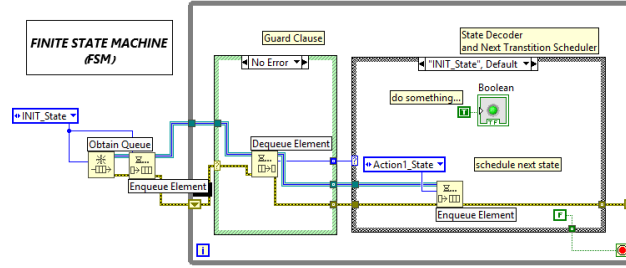


Figure 14: Finite State Machine (FSM) Example Implementation in LabVIEW

Producer-Consumer LabVIEW Design Pattern

The second paradigm employed, upon which both **main.vi** and the subVI **Arduino_FlowPump_Controller.vi** are based, is the **Producer-Consumer (Event-Driven)** design pattern [11]. This approach is particularly well-suited for real-time applications that must respond promptly to user-generated events through the graphical user interface (GUI), while simultaneously ensuring that the program remains non-blocking—that is, each event is handled asynchronously and independently from others.

This architecture consists of an event-handling structure, referred to as the 'producer', which monitors all relevant user-GUI interactions based on the software's functional requirements. The producer defines the behavioral response for each incoming event, but is unaware of the actual implementation details necessary for event handling. These are delegated to a separate structure known as the 'consumer'. Once the consumer receives a command from the producer, it executes the corresponding operations necessary to complete the event-handling cycle.

Figure 15 illustrates the block diagram implementing a finite state machine alongside the event-based Producer-Consumer pattern. The events correspond to user-GUI interactions. Each relevant interaction triggers a state transition within the finite state machine, which is implemented in the consumer loop using an enumerated constant to manage state flow.

In this project, a multi-consumer architecture was adopted, where each consumer is designed to respond to events of a different "nature"—that is, events that require access to different software or hardware resources.

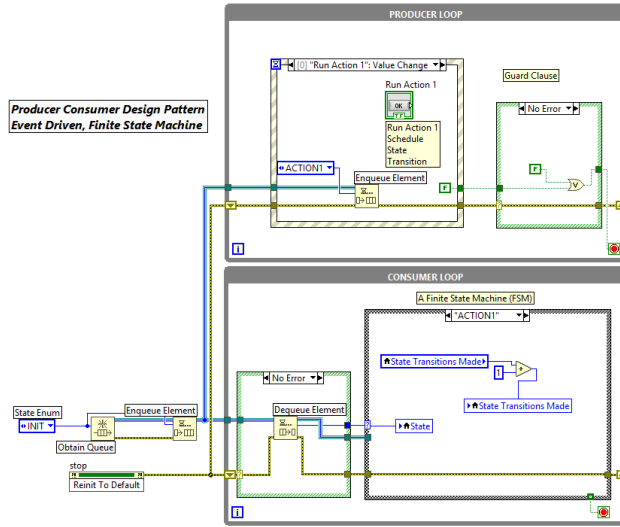


Figure 15: Producer Consumer Design Pattern Example Implementation

main.vi



The Virtual Instrument (VI) used for calibrating the analog sensor and with which the user primarily interacts is named *main.vi*. This VI does not directly manage the connection with the measurement instruments in the system; rather, it orchestrates a set of subroutines (or **subVIs**), shown in Fig. 13, which in turn, through the libraries available in LabVIEW, acquire measurement samples and convert the control commands addressed to the instruments into the correct syntax for the respective communication interface.

Specifically, two distinct types of consumer structures were implemented: one to manage the events that trigger state changes in the devices controlled via the Serial and GPIB interfaces—namely the Arduino UNO board and the power supply unit—and a second consumer dedicated to handling the NI DAQ-6008 data acquisition device, which samples the voltage measurements from the analog sensor to be calibrated. Each of these event handlers internally implements an additional architectural paradigm—namely, the finite state machine based on queued state scheduling (Queued State Machine). In general, finite state machines are used to model the behavior of systems in which, for a given input, the output depends on the system's current state. In the context of this project, states are used to model the connection status between the main program (LabVIEW) and the devices controlled via their respective interfaces. For example, a given input, such as reading flow measurements from the

digital sensor, will only produce valid data if the connection to the Arduino board has been successfully initialized. Similarly, a request to output a specific current from the power supply to drive the air pump can only be fulfilled if the requested current does not exceed the limits configured on the instrument. Both of these constraints—the validity of the Arduino UNO connection and the upper current limit—define a state for their respective devices.

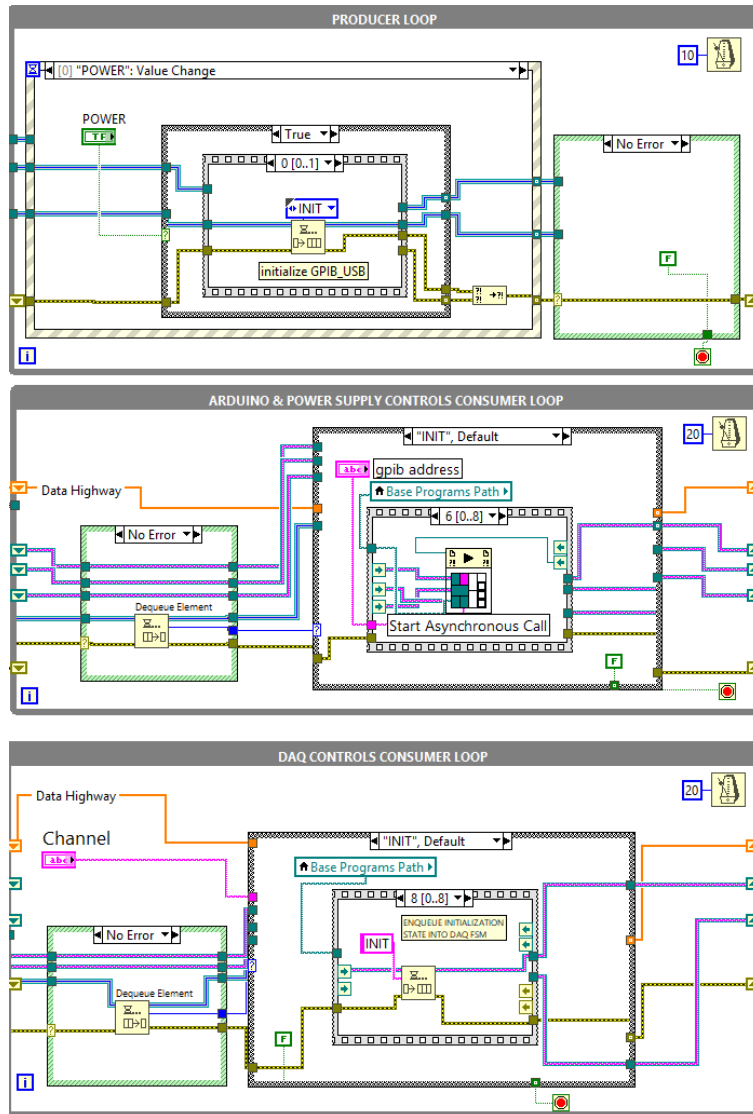
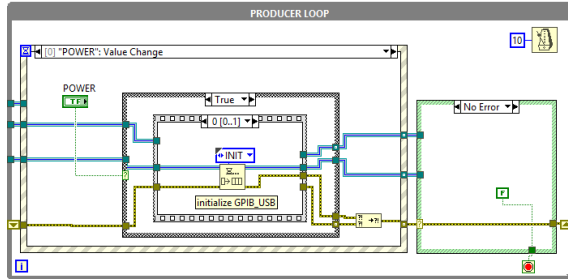
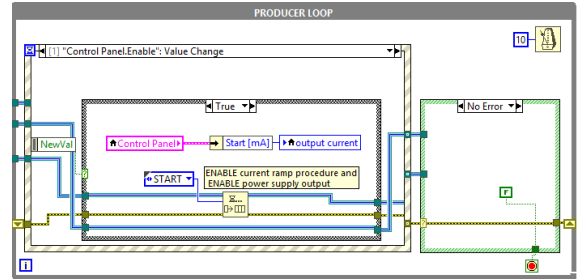


Figure 16: *main.vi* Producer-Consumer Queued State Machine

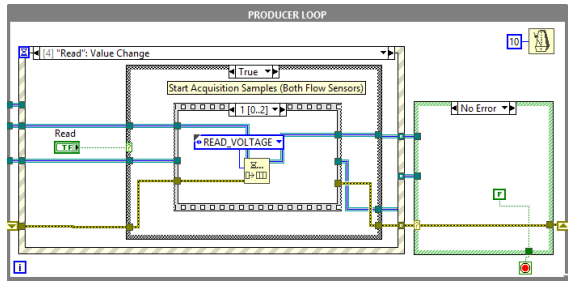
Producer Loop



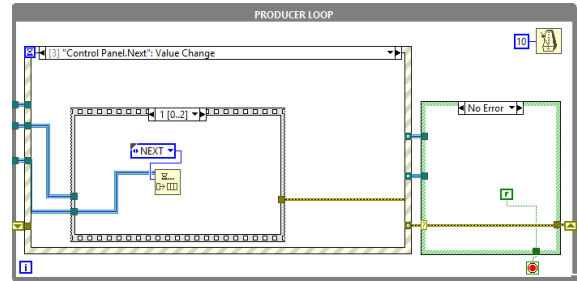
(a) When the **main.vi** is run the Producer Schedules the Initialization State in each Consumer Loop



(b) When *Enable* is pressed on the GUI, the Current Ramp is activated in the Start state



(c) When *Read* is pressed on the GUI, the signal acquisition from both sensors is activated



(d) When *Next* is pressed output current changes by one step up (or down, depending on the state)

Figure 17: A collection of four event handling cases, demonstrating how the Producer schedules state transitions for the Consumer loops that run in parallel.

The internal structure of the producer loop includes an event-handling structure where the programmer defines a set of cases to respond to specific events. Notification from the producer to the consumer loops about an event is achieved through the use of queues. It is important to note that the producer is placed in parallel with the consumers, specifically stacked vertically above them, which is essential for the correct execution of the program. Indeed, if the loops were placed side by side, due to the way graphical programming in LabVIEW is designed, the consumer loops would not be correctly executed. This is shown in Figure 16. The queues used in the producer-consumer communication ensure that all events are properly received and processed by the consumer loops. Four different event cases are shown in Figure 17.

The first case is activated at the start of the **main.vi** execution, to command the two consumer loops to enter initialization states. Figure 17a shows the use of the first consumer loop **queue reference** and the **INIT** constant Enum value used to represent the state transition. This is all inserted in a stacked

sequence to minimize space occupied. The second frame would show the same state scheduling, this time for the second loop, which handles the DAQ connection.

The second, third and fourth cases are very similar and also related to each other, in the sense that they usually execute in order since these cases refer to the push buttons the user interacts with to control the **Current Ramp** used to generate an ascending and descending current values input sequence to the air-pump to generate various air-flows.

Further, to handle potential errors during program execution, each producer/consumer structure includes a **Guard Clause**—a case structure that changes color (green or red) depending on whether the connected terminal contains an error-free cluster or a cluster with an error code. When the error cluster contains an error (identified by the corresponding alphanumeric code), the border of the structure turns red. In this case, an error signal is sent to the consumer loops, which safely terminate their execution, releasing any resources previously acquired. If no error is present, execution continues with the Producer loop returning to its listening state, while the consumer loops proceed with their respective state transitions.

Finally, notice that, since the producer loop must run fast enough to respond promptly to GUI-driven user events, an iteration delay of 10 ms was introduced using the **Wait for Next Multiple ms** function, with a constant value of 10.

Arduino & Power Supply Controls Consumer Loop

The first consumer loop encountered in the main.vi implements a finite state machine whose states represents important steps into the control of the communication to the Arduino board and the power supply controlled via GPIB. The state diagram of this consumer loop is presented in Figure 18. The quoted strings represent the names of controls on the front panel of the main.vi which, when their values change, produce an event which the producer loop converts into a state transition for this FSM.

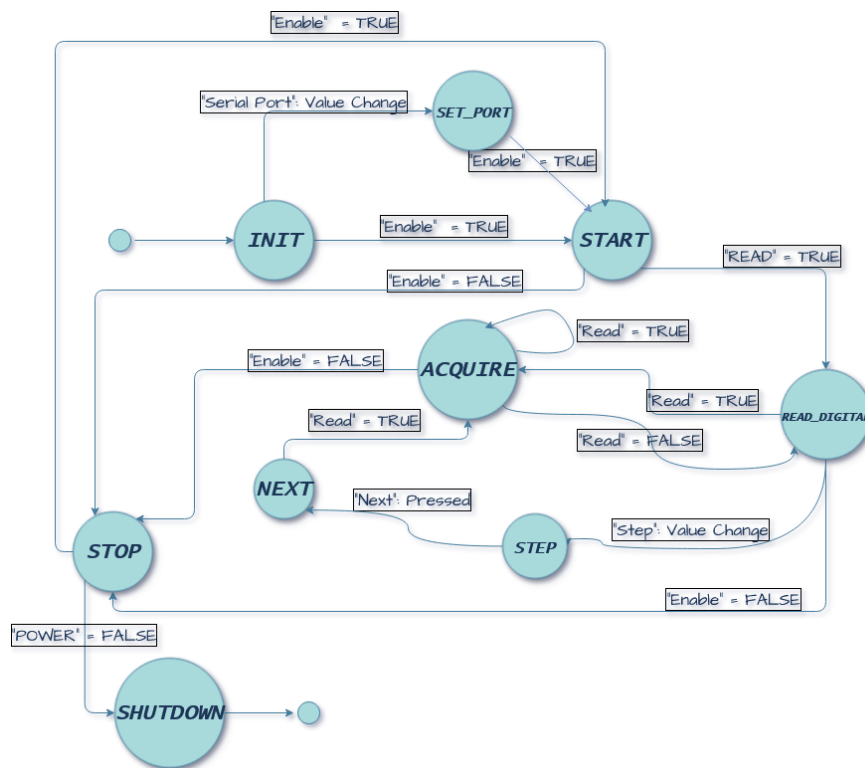
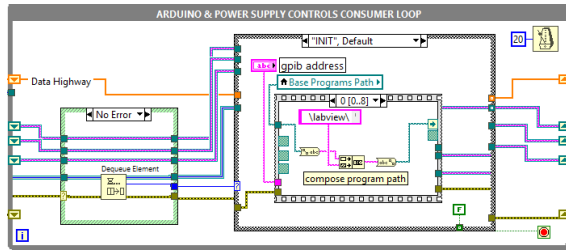
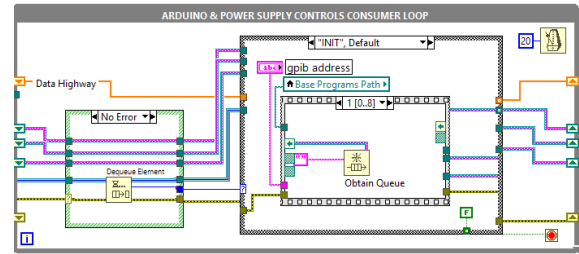


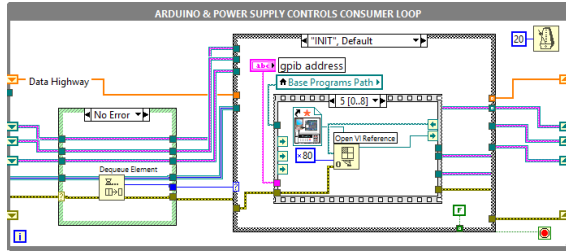
Figure 18: State Diagram of the "Arduino & Power Supply Controls" consumer loop in main.vi.



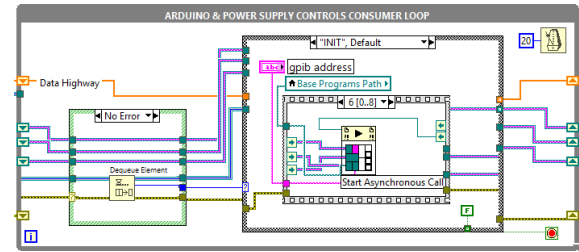
(a) Arduino & Power Supply Controls Consumer Loop: INIT state, frame 0



(b) Arduino & Power Supply Controls Consumer Loop: INIT state, frame 1



(c) Arduino & Power Supply Controls Consumer Loop: INIT state, frame 5



(d) Arduino & Power Supply Controls Consumer Loop: INIT state, frame 6

Figure 19: A collection of four frames from the sequence of operations that execute in the INIT state within the first consumer loop. The loop initializes the necessary controls to start the subVI **Arduino_FlowPump_Controller.vi** asynchronously

To handle user-GUI interactions related to the control of the Arduino UNO board and the power supply for the air pump—communicated respectively via the Serial and GPIB interfaces—a single consumer loop was implemented. Although this design choice may seem counterintuitive, given that the two communication channels are independent of each other, it was necessary because both connections are managed by a dedicated subVI named **Arduino_FlowPump_Controller.vi**. This subVI, in turn, calls two other subVIs, **Agilent_GPIB_FSM.vi** and **Arduino_Serial_FSM.vi**, which control the GPIB and Serial interfaces separately, establishing a hierarchical structure.

Asynchronous Call

By default, subVIs in LabVIEW are executed synchronously. In this mode, the calling VI halts its execution while the subVI performs its assigned task—defined by its API—and resumes only when the subVI returns control, potentially along with output data. This execution model adheres to the principle of *information hiding*, which promotes encapsulation and modularity by abstracting implementation details. However, synchronous execution forces the caller to wait, which may be undesirable in systems

requiring responsiveness or concurrent processing.

In contrast, an asynchronous call allows the calling VI to continue executing independently of the subVI. This feature is particularly useful for launching background processes—such as remote instrument control via Serial or GPIB—without blocking the main VI. User-GUI interactions are meanwhile collected into a queue; once the asynchronous subVI is launched, each interaction is handled in the order it was received, with no disruption to the main application. From the caller’s perspective, the subVI’s internal execution remains entirely opaque and self-contained.

In our system, asynchronous calling was employed to launch the subVI **Arduino_FlowPump_Controller.vi**, which manages the interface to the Arduino UNO board and the power supply unit. This subVI is started from within the consumer loop that handles instrument communication.

Figure 19 provides an overview of the main components involved in setting up the asynchronous call:

- **(a)** shows the generation of the *file path* that is used for pointing to the memory location on the user workstation to **Arduino_FlowPump_Controller.vi**.
- **(b)** illustrates the creation of one of the three queues used for: (1) sending control commands, (2) transferring sensor samples acquired by the Arduino, and (3) communicating the current setpoints for the power supply.
- **(c)** presents the use of the **Open VI Reference** function, which prepares the VI reference required for launching the subVI.
- **(d)** shows the invocation of the **Start Asynchronous Call** function, which executes the subVI in a non-blocking fashion.

The asynchronous mechanism is based on two core functions from the *Application Control* palette:

1. **Open VI Reference.vi**: This subVI generates a reference to the target VI to be executed asynchronously. Three terminals must be configured:
 - **type specifier VI Refnum (for type only)** – connected to a **Strictly Typed Static VI Reference**, which is a prerequisite for asynchronous invocation.
 - **application path** – a *path* control pointing to the file location of the subVI.

- **options** – a hexadecimal constant that specifies the execution mode. Setting this terminal to **x80** indicates a **Call and Forget** mode, meaning the subVI is launched without collecting any output.
2. **Start Asynchronous Call.vi**: This subVI starts the asynchronous execution of the VI reference generated above. When configured correctly, it displays the subVI's input/output terminals, allowing parameter wiring. It also returns a reference to the launched subVI, which can be retained by the caller in case termination or interaction is required later.

In our application, the strictly typed reference to **Arduino_FlowPump_Controller.vi** is connected to its respective inputs: the GPIB address of the power supply unit, the path to the resources folder, and three queues passed by reference. These queues serve the following functions:

- **Command Queue**: sends control commands to manage the connection states.
- **Sensor Data Queue**: transfers the digital sensor data sampled by the Arduino, ensuring no samples are lost (*lossless communication*).
- **Current Setpoint Queue**: passes the real-time current output value in mA generated the power supply.

This configuration allows the subVI to run concurrently in the background while the main VI remains responsive to further user interaction and other tasks.

DAQ Consumer Loop

The second type of consumer loop, which handles data acquisition using the NI DAQ-6008 board, implements a pattern identical to that described for Consumer 1. Upon initialization of **main.vi**, a dedicated queue is created, and its reference is subsequently used by the producer loop to control state transitions exclusively for events related to data acquisition via the DAQ device. As previously described, the states are represented individually by numerical constants, which via an enum control can be referenced to as strings which represent either commands to be carried out by the DAQ board or a change in the parameters of the device, as seen in the state diagram in Figure 20. Note that the arrows on a state transition which do not have any condition, indicate transitions that happen automatically.

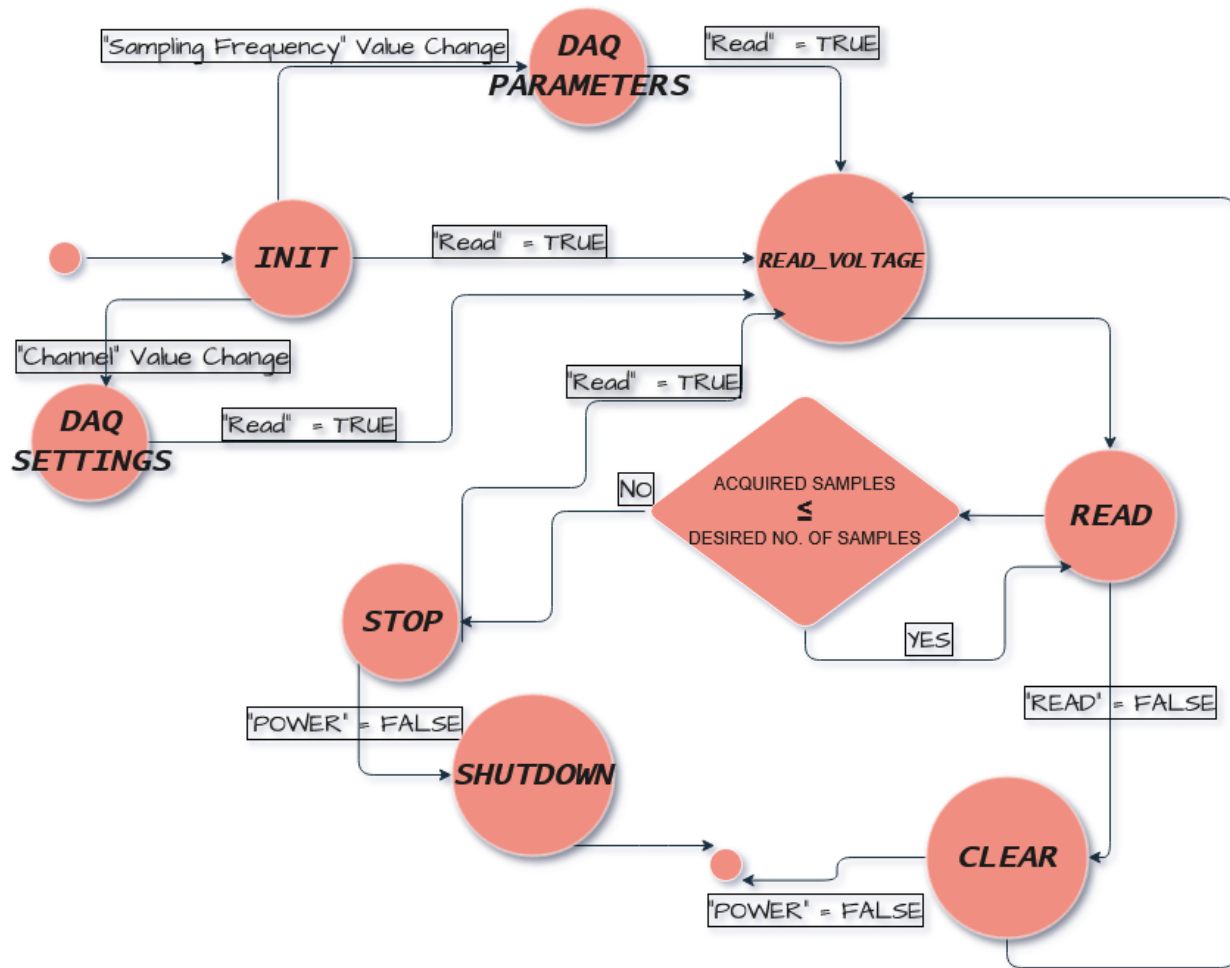
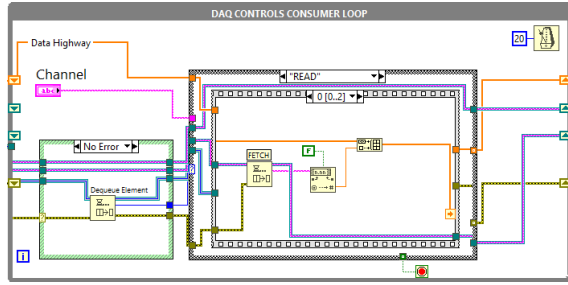
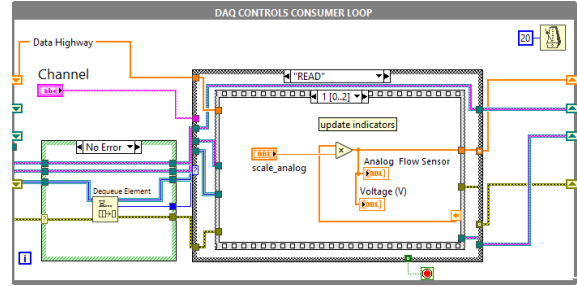


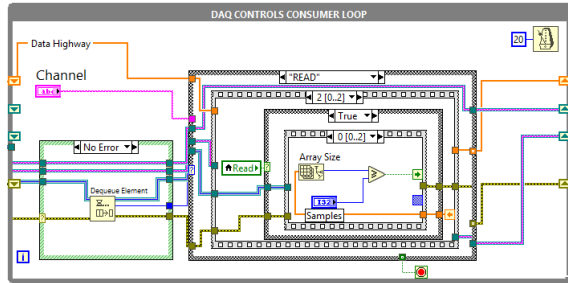
Figure 20: State Diagram of the "DAQ Controls" consumer loop in the main.vi. The DAQ board connection is managed by the *daq_fsm.vi* subVI..



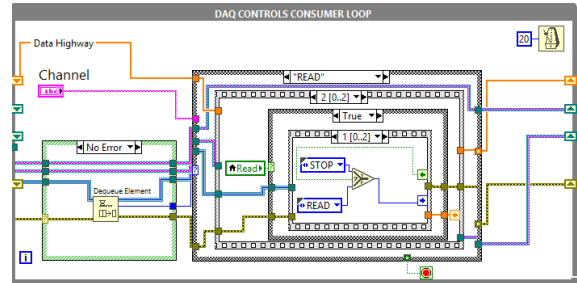
(a) DAQ Controls Consumer Loop: READ state, frame 0



(b) DAQ Controls Consumer Loop: READ state, frame 1



(c) DAQ Controls Consumer Loop: READ state, frame 2 (I)



(d) DAQ Controls Consumer Loop: READ state, frame 2 (II)

Figure 21: A collection of the frames that executes in sequence when the DAQ consumer loop enters the READ state. This happens whenever the user requests to acquire new samples during a calibration procedure run, by pressing the READ button on the front panel.

Among the few states that model the connection with the DAQ device, the one that deserves more attention is the READ state. Whenever the user requests a new data acquisition from both sensors by pressing the *READ* button on the front panel, the Producer schedules a **read state** into both consumer loops.

Figure 21 provides an overview of the main components involved in reading data from the measurement queue and defining the next state transition:

- **(a)** shows how the consumer loop fetches a new voltage sample acquired and transferred by the DAQ FSM subVI presented later. The sample is first **dequeued** by the READ state and then converted into a double numeric value. Finally, the sample is concatenated with previous samples already drawn out of the queue (if any).
- **(b)** illustrates the updating process of the indicators shown on the front panel. Samples are multiplied by a given factor to match the desired measurement unit requested by the user.

- **(c)** shows the comparison between the requested number of samples to be acquired (specified on the front panel by a numerical control that the user modifies before acquisition starts) and the current number of samples drawn from the measurement queue.
- **(d)** result of the previous comparison controls the next state transition. Whenever enough samples are acquired, the next state transition is set to be into the STOP state, which stops acquisition by clearing the **DAQmx Task**. Instead, if the number of samples is less than the desired value, the next state will still be the READ state.

Notice that the READ state executes and reschedules itself until the number of samples acquired matches the desired value expressed by the user on the front panel. Nevertheless, the user can stop the READ state in the middle of the acquisition process by deactivating the *READ* button on the front panel, shown in Figure 22. Whenever that happens, the Producer enqueues the STOP state into the second consumer loop state queue, hence stopping the acquisition procedure.

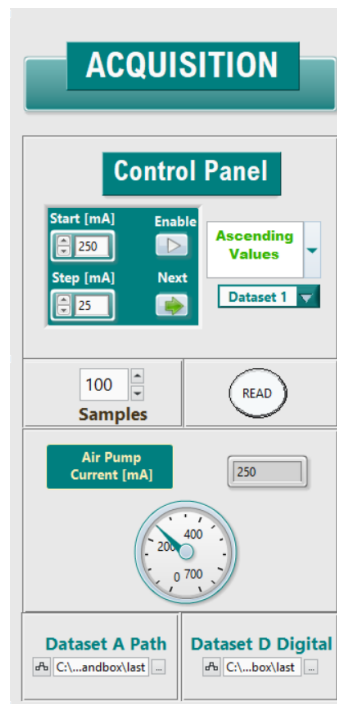
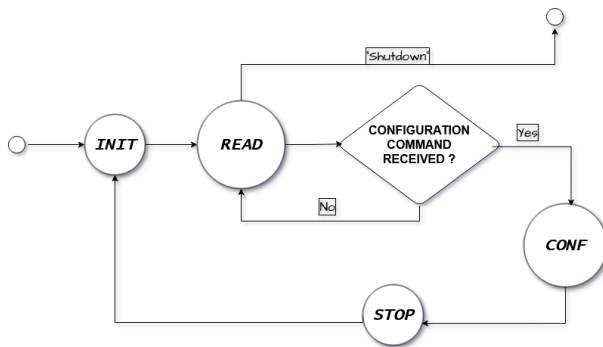


Figure 22: Acquisition Tab in main.vi shows the **Control panel** and the many interactive controls such as the *READ* button, the *Next* button and the *Samples* control.

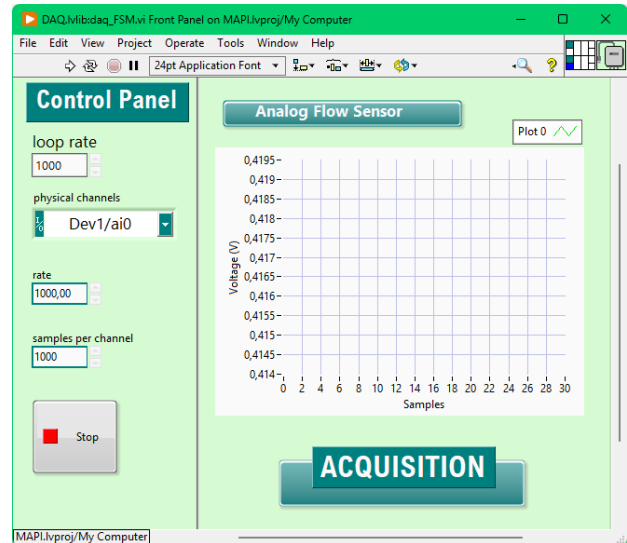
daq_fsm.vi



The subVI *daq_fsm.vi* is designed to manage the interconnection with the data acquisition (DAQ) board, which allows sampling of the output signal from the conditioning circuit. The latter, described in its dedicated section in this manual, receives as input—among other things—the two output signals from the analog sensor, that is, a pair of voltages generated by the thermopiles composing the sensor. The conditioning circuit then computes the difference between these two voltages and amplifies the result by a gain factor. The resulting signal is then connected to one of the pins of the DAQ board to be sampled and digitized. The state diagram and the front panel of this subVI are shown in Figure 23:

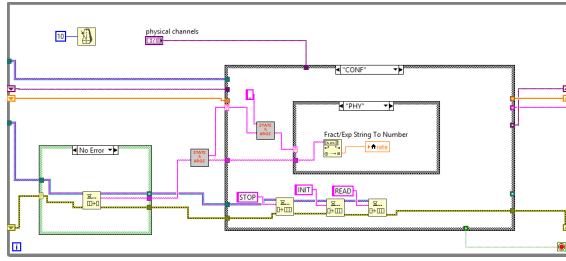


(a) State Diagram of "daq_fsm.vi" subVI

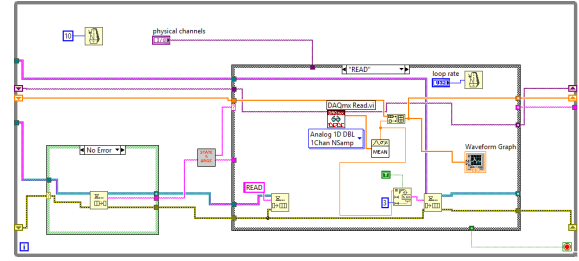


(b) Front panel of "daq_fsm.vi" subVI

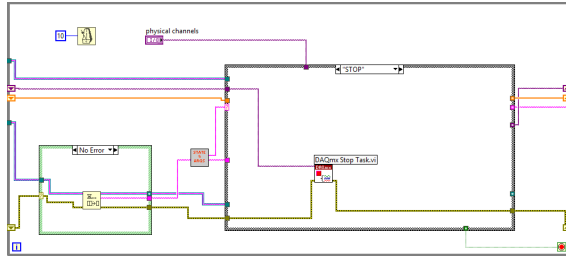
Figure 23: **histogram.vi** subVI used to create all the histograms for each feature column in a given dataset



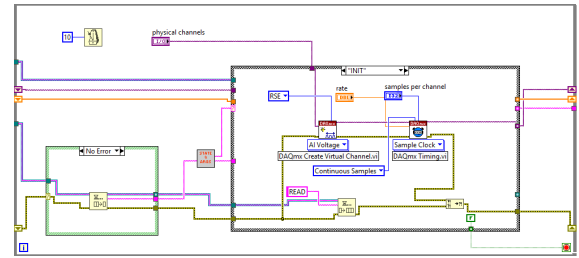
(a) daq_fsm.vi: CONF state



(b) daq_fsm.vi: ASYNC state



(c) daq_fsm.vi: STOP state



(d) daq_fsm.vi: INIT state

Figure 24: A collection of the frames that executes in sequence when the DAQ consumer loop enters the READ state. This happens whenever the user requests to acquire new samples during a calibration procedure run, by pressing the READ button on the front panel.

The DAQ board connection, via USB, leverages *National Instruments (NI)* LabVIEW drivers to facilitate communication with acquisition hardware. These drivers provide libraries for signal generation and acquisition on *virtual channels*, which are abstractions of physical I/O points managed by *Tasks*.

Fundamentally, **daq_fsm.vi** implements a finite state machine (FSM) that models the lifecycle of a virtual channel. Its default, initialization state (Fig. 24d) creates and configures the virtual channel for analog voltage input (*AI:Voltage*), specifically as *RSE* (Referenced Single-Ended), where the measurement system shares a common ground. This state also sets default acquisition parameters via the *DAQmx Timing* task, defining sample count, continuous acquisition mode, and sampling frequency. The initialization state then transitions to the *READ* state.

In the *READ* state (Fig. 24b), the *DAQmx Read.vi* automatically commences acquisition, using the pre-configured parameters to sample the input signal. "Samples per channel" defines the acquisition board's internal buffer size, while "rate" specifies the DAQ sampling frequency. With *Continuous Acquisition*, data is continuously buffered, overwriting older data in the circular queue if space is insufficient.

Crucially, the **loop rate** control determines the repetition frequency of the READ state’s ‘while’ loop. This timing directly impacts the program’s ability to read data swiftly enough to prevent buffer saturation and data overwrite.

From the READ state, three transitions are possible: to the *STOP* state (Fig. 24c) to halt the task, back to the initial initialization state, or back to the *READ* state itself.

Arduino_FlowPump_Controller.vi



The subVI *Arduinio_FlowPump_Controller.vi*, which is part of the *GPIB_UART.lvlib* library. This subVI controls the connection with the Arduino UNO and the programmable Power Supply Unit (PSU)- which in turn controls the air-pump- via the *Serial* and *GPIB* interfaces, respectively. As before, the presence of separate consumer loops stems from the need to simultaneously manage two fundamentally different types of connections, each of which must be controlled independently without interfering with the other. These two connections refer to the GPIB interface, used to control the Agilent e3634A power supply for driving the air pump, and the UART serial interface, employed to manage the acquisition of measurements from the digital sensor via the Arduino UNO board. Hence, the subVI calls two other subVIs that handle these connections: **Arduino_Serial_FSM.vi** and **Agilent_GPIB_FSM.vi**.

As in the previous case, the producer loop is structured as a while loop containing an event structure, followed by a guard clause designed to handle potential malfunctions either at the current level or in the lower layers (i.e., the consumer loops governed by state transition programming).

Since each event that triggers the producer loop is associated with a descriptive label, the reader is referred to the block diagram documentation for further details on how each event is handled and which consumer loops are involved.

Arduino_Serial_FSM.vi



The firmware on the Arduino UNO’s *ATmega328P* microcontroller is responsible for acquiring, converting, and transmitting flow measurements from the calibrated digital sensor. This read-convert-transfer cycle initiates upon the first successful PC-Arduino connection, which requires prior configuration of the serial port and baud rate. Once active, the serial connection allows the user to read and visualize real-time airflow data. The sampling frequency can also be modified dynamically, provided the con-

nection is active. Users can terminate and re-establish connections to adjust parameters like serial port or baud rate.

These interaction requirements are modeled as states within a finite state machine (FSM), implemented by the **Arduino_Serial_FSM.vi** subVI from the **GPIB_UART.lvlib** library. The *Serial Connection Consumer Loop* asynchronously initiates this subVI and manages state transitions by sending high-level commands via a dedicated queue whenever the user requests a connection state modification. This abstraction, depicted in Figure 25, means the loop operates without directly engaging low-level functions like VISA.

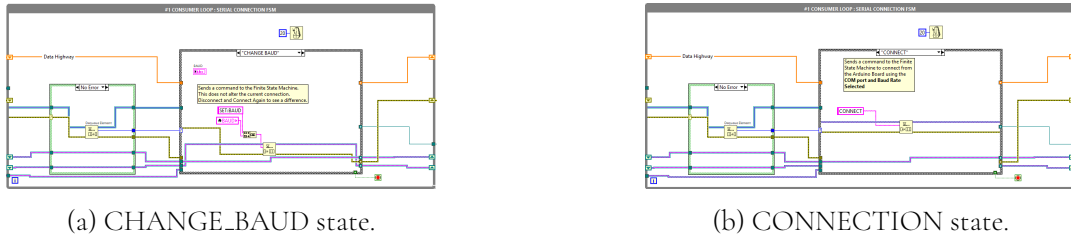


Figure 25: *Serial Connection FSM* consumer loop in *Arduino_FlowPump_Controller.vi* subVI.

Separating the logic that handles state transitions from the implementation of the operations performed within each respective state promotes a clear distinction between high-level and low-level program layers, fostering modularity and independence between them.

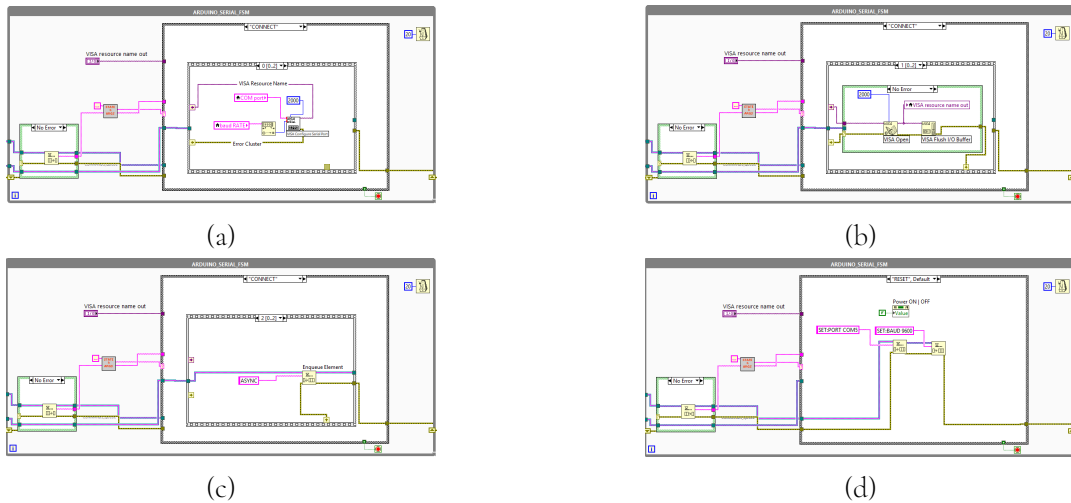


Figure 26: *Arduino_Serial_FSM.vi* Block Diagram.

The subVI named **Arduino_Serial_FSM** implements the most elementary form of a finite state machine (FSM) within the graphical programming paradigm of LabVIEW. To function properly, the subVI

requires two queues: one for the data read from the Arduino board and another for the commands issued by the calling subVI. Additionally, an error cluster must be connected to ensure proper error handling.

The key difference between this FSM implementation and the one used in the consumer loops previously described lies in the representation of the finite automaton's states. Here, the states are not represented by customized *enum* constants, but rather by alphanumeric strings. This allows each state to be expressed as a string of characters. Furthermore, each state—interpreted as a command to be executed—can be associated with an input, or argument, that enriches the informational content of the state transition being executed.

The drawback of this approach is that states are not as easily selectable as in the case of custom controls, where the next state can be chosen from a drop-down menu. Moreover, using strings introduces the possibility of typographical errors during programming, potentially leading to transitions toward non-existent states (e.g., intending to transition to state "READ", but instead typing "RED"). This risk can be mitigated by defining a "safe" default state—one that does not produce any unintended effects regardless of the current state. In the case of the subVI described here, this default state corresponds to the "RESET" state, as illustrated in Figure 26d. Although this state reinitializes the resources and parameters used for the connection, it does not alter the current connection, since these modifications do not affect the active resources and parameters but only those that might eventually be used for a new connection after the current one is terminated. The state diagram for this subVI FSM is shown in Figure 27:

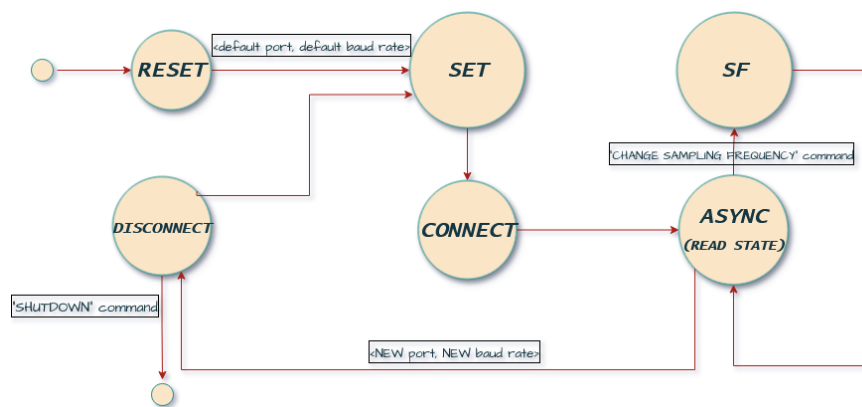


Figure 27: Finite State Machine State Diagram of "Arduino_Serial_FSM.vi" subVI started asynchronously in Arduino_FlowPump_Controller.vi subVI



(a) State_and_args.vi SubVI Description

(b) State_and_args.vi SubVI Block Diagram

Figure 28: State and Args subVI used to decode the next state transition in the Arduino_Serial_FSM.vi

In the queue used to control state transitions in the finite state machine implemented by the subVI managing the connection to the Arduino board, strings are inserted in the following format:

COMMAND (delimiter) Argument

To decode the content of these strings, a dedicated subVI has been developed. This subVI accepts two input strings: the string to be decoded and the delimiter that separates the state (i.e., the command) from its arguments (i.e., input parameters). It returns two output strings, corresponding to the parsed command and the recognized parameters.

The connection to the Arduino board via the serial interface is established using the parameters selected by the user, which are used to configure the *VISA Configure Serial Port* function, as shown in Figure 26a. If the port is successfully initialized, a new session is opened using the *Open Session* function, which returns a unique identifier for the established connection. Since the next state is automatically programmed to perform asynchronous reading from the microcontroller, the memory buffer is cleared to ensure that the program will read only the most recently acquired data.

By asynchronous reading, we refer to the process of reading from a memory area without blocking program execution until data becomes available from the producer (i.e., the Arduino board). In this implementation, although the current state re-enqueues itself in the state transition queue, a different state may still be scheduled by the calling subVI, namely *Arduino_FlowPump_Controller.vi*. If this occurs, since the queue operates under a FIFO (First-In-First-Out) policy, the current state will be executed once more before proceeding to the newly scheduled one (e.g., "DISCONNECT"). The subsequent state might not automatically reprogram the asynchronous reading state, in which case the continuous reading from the buffer may be interrupted. In the ASYNC state, shown in Figure 29, the program uses the **VISA Read** function from the *Instrument I/O / VISA* subpalette to read data from the serial connection.

Then, it enqueues the data acquired (one sample at a time) in the queue used by the higher-level subVI Arduino_FlowPump_Controller, specifically by the first consumer loop.

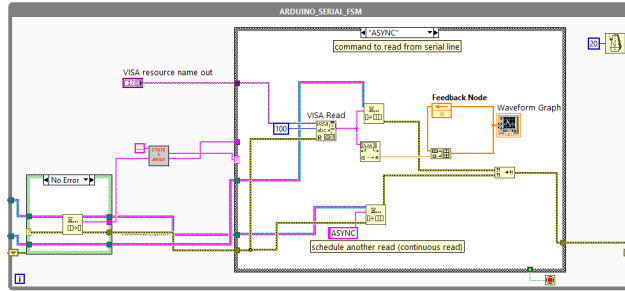


Figure 29: Asynchronous Read state used to read measurements asynchronously from Arduino board

Agilent_GPIB_FSM.vi



The connection used to control the power supply unit (PSU) driving the air pump is established via the GPIB interface. Consequently, the consumer loop handling events that affect the state of the remotely controlled instrument must use commands defined by the protocol, and specifically, by the PSU's programming manual. The table below lists the GPIB commands used throughout this project, along with their corresponding functions. For further details on this subject, we refer the reader to the instrument's programming manual.

Command or Query	Example	Description
OUTPut	OUTPUT ON	Turn on the output of the instrument
CURRent	CURR 0.1	Change the current value of the current output to 0.1A
VOLTage:RANGe	VOLT:RANG P25V	Select the +25V output channel
CURRent:PROTection	CURR:PROT 0.7	Set limit current value to 0.7A
*RST	*RST	Reset instrument to default settings
SYSTem:ERRor?	SYST:ERR?	Read any errors from the error buffer
VOLTage	VOLT 12	Change the voltage value of the voltage output to 12V

Table IV.1: Remote PSU commands.

The commands and queries are used within the subVI Agilent_GPIB_FSM.VI. This subVI is launched via an asynchronous call from the second consumer loop within Arduino_FlowPump_Control, which writes high-level commands in human-readable format, and the low-level subVI translates these commands in the proper syntax used by the PSU. The second consumer loop does not know how these

commands are transferred to the instrument, once again highlighting the advantage of the information hiding technique.

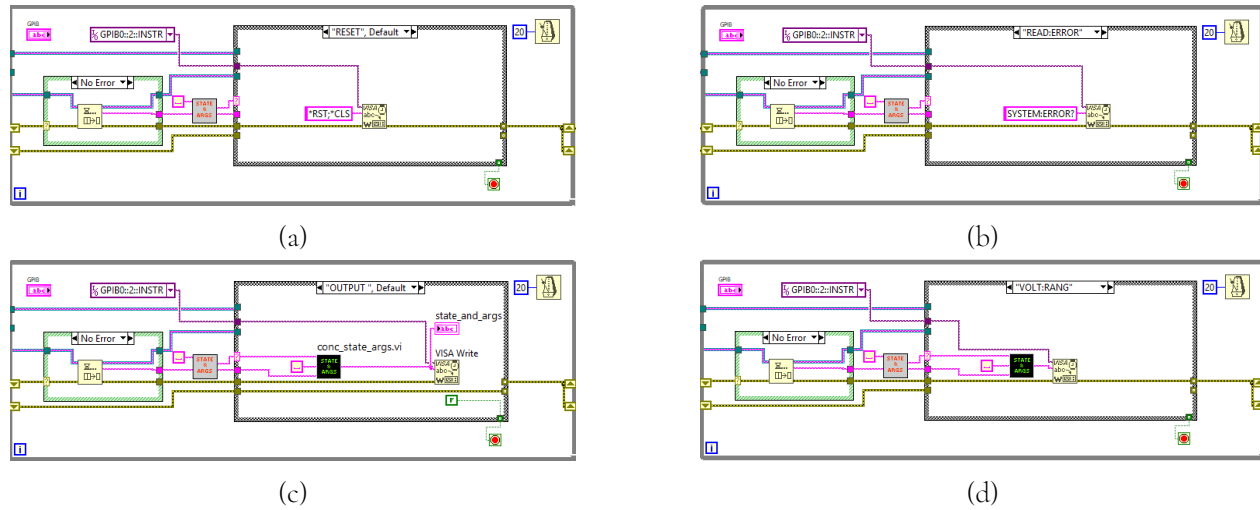


Figure 30: Agilent_GPIB_FSM.vi Block Diagram.

The main functions used from the Instrument I/O / VISA subpalette. VISA subVIs are able to transfer the command desired as prescribed by the interface used, which is identified based on the **VISA Resource Name** used. In this case, since the program uses a **GPIB** resource name, the VISA subVI transfers the commands according to the GPIB protocol, as shown in Figure 30d. Also, note that the low-level subVI uses a **support VI** called *conc_state_arg*. This subVI reconstructs the command string used by the higher-level subVI *Arduino_FlowPump_Control.vi*.

Current Ramp FSM

The third while loop in the *Arduino_FlowPump_Control.vi* implements the finite state machine required to model the states of a current ramp that is applied to the air pump, generating the airflow to be measured by the transducers.

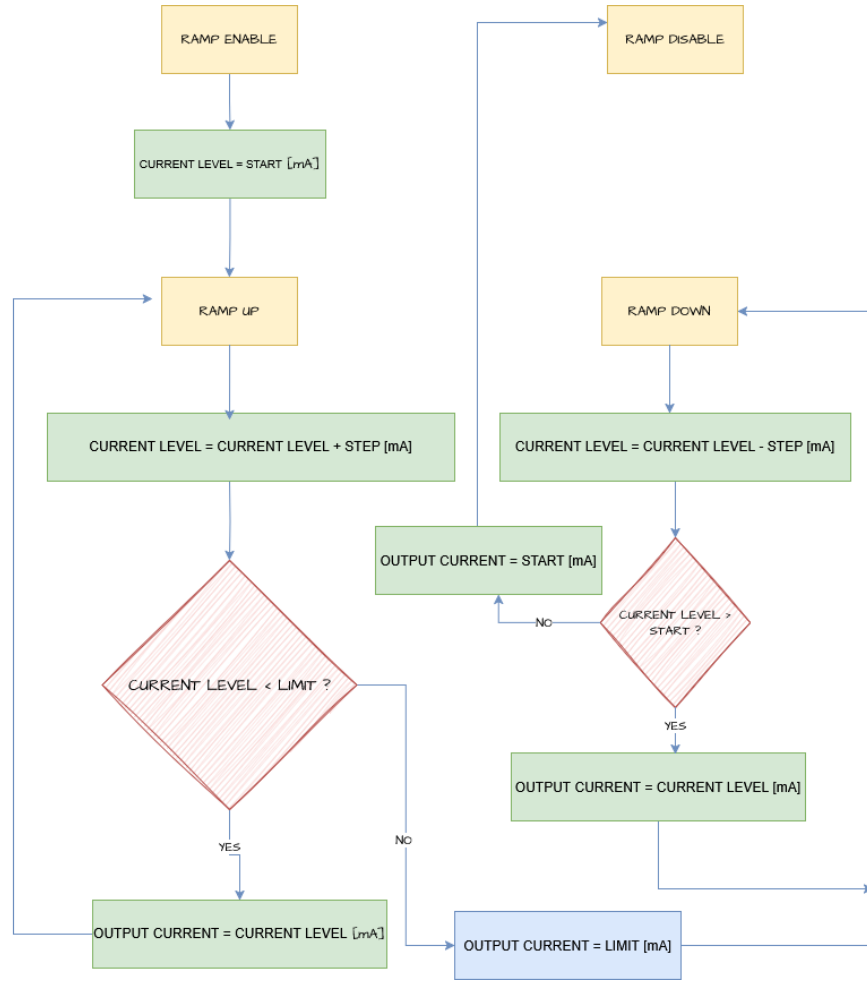
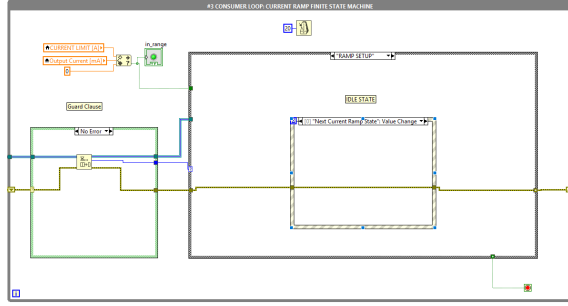


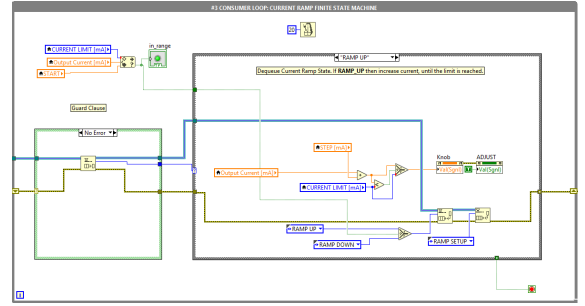
Figure 31: Current Ramp Flow diagram

The diagram shown in Figure 31 effectively illustrates the process of generating a current ramp, but it is not complete in itself, as some states are intentionally omitted for simplicity. However, these omitted states are essential for the correct functioning of the program in practice.

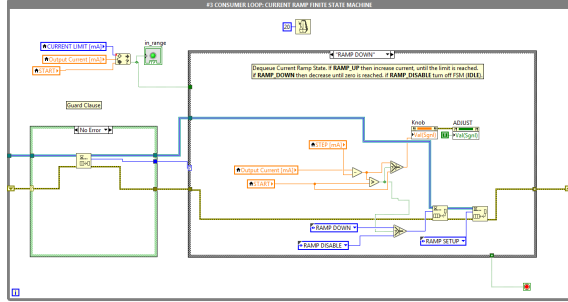
The decision to implement a finite state machine stems from the intent to make the ramping process both manual and iterative. Indeed, if one were to implement a program that strictly followed the flow diagram shown in Figure 31, the execution would proceed automatically and without user intervention, as such a process typically requires no external control.



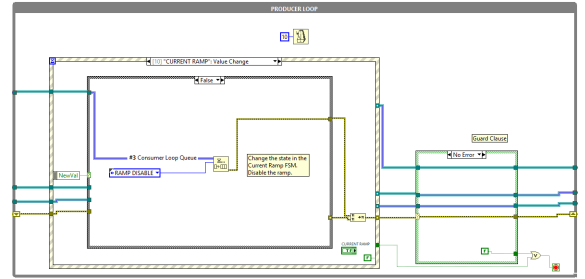
(a) RAMP SETUP state



(b) RAMP UP state



(c) RAMP DOWN state



(d) RAMP DISABLE state

Figure 32: Current Ramp FSM in the Arduino_FlowPump_Control subVI.

In contrast, our design aims to let the user decide when to apply the next current value ($CURRENT\ VALUE \pm STEP [mA]$), depending on the state which could either be *RAMP UP* or *RAMP DOWN*), rather than having the algorithm advance automatically. Therefore, a key state—absent from the diagram in Figure 31—must be introduced: a state in which the system waits for a user-issued command before transitioning to the next state, based on a condition evaluated in the previous stage. In the program, this waiting state is referred to as **RAMP SETUP**, shown in Figure 32a.

When the automaton is in the **RAMP UP** or **RAMP DOWN** states—that is, when increasing or decreasing current values are being generated, respectively—two transitions are scheduled. One is conditional, based on a Boolean evaluation; the other transitions to the waiting state. These states are shown in Figure 32b and 32c, respectively.

To implement this mechanism, an event structure similar to the one seen in the Producer loop is inserted into the case structure within the third consumer loop. The difference lies in the fact that we wait for one of two events to occur:

- The user disables the current ramp generation procedure (**RAMP DISABLE**);
- The user requests that the next current value be generated according to the current state, the

current limit, and the selected step.

In the first of these two cases, it is the Producer loop that inserts the current ramp disable state into the state transition queue, as shown in Figure 32d.

Commands Decoder Loop

Finally, we added a fourth while loop in the `Arduino_FlowPump_Control.vi`, which can be used with a queue initialized through the subVI's terminals, to make the subVI itself programmable by other VIs created by the user. We have indeed defined an **API** with which the user can send strings (*commands*) accompanied by arguments (*parameters*) that generate an event, effectively simulating a real user pressing a push-button or any other possible user-GUI interaction on the front panel. This allows the **main.vi** program to modify the state of the controls on the subVI's front panel without necessarily interacting with the virtual instrument's front panel. Furthermore, this modification allows for an additional level of abstraction; in fact, we have defined commands that involve multiple actions (i.e., multiple interactions between the user and the GUI) that allow more operations to be performed with a single command.

Calibration

Calibration Procedure Loop

To aid the user in the analysis of the datasets acquired and produce a series of results that can be used to produce a calibration certificate, a set of VIs was developed for this aim. These subVIs are used within the **main.vi** block diagram, and are executed once the user loads the datasets in the application after the data acquisition phase. In the main.vi a fourth while loop executes whenever the user loads the acquired datasets during the current ramp and data acquisition procedures. In total, the user must acquire two datasets for each phase (ascending and descending input sequence) for both sensors.

A set of two dedicated **LabVIEW libraries** was created to group the subVIs related to a specific task in the calibration procedure task: **statistics.lvlib** and **CALIBRATION.lvlib**. The subVI Icons are shown in Figure 33.

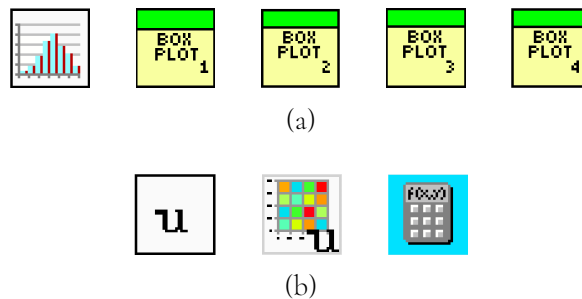
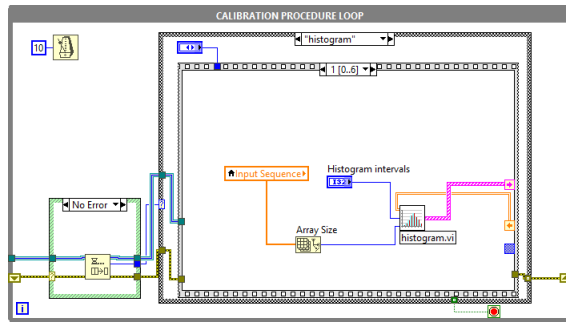


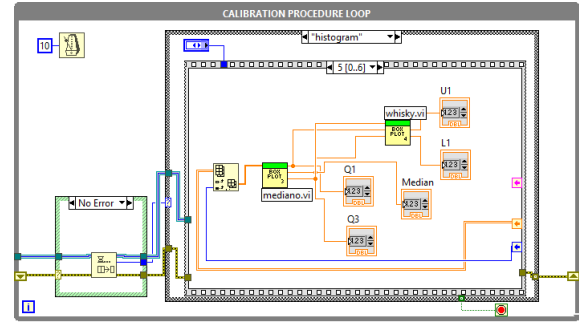
Figure 33: (a) statistics.lvlib subVIs. (b) CALIBRATION.lvlib subVIs.

Histograms: Samples Statistical Distribution

Before evaluating Type A Uncertainty, it is essential to check the probability distribution of the sample data in each dataset, as this helps determine whether assumptions underlying statistical models—such as normality—are valid [1].



(a) Evaluation of the histograms from the datasets loaded by the user after acquisition

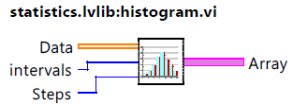


(b) Search for median, lower and upper quartiles (Q1, Q3) and whiskers (U1, L1).

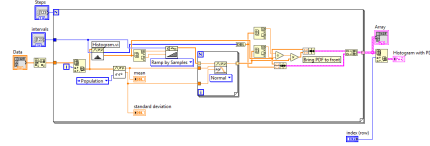
Figure 34: Calibration Procedure Loop: histogram and boxplots graphs and statistical metrics computation.

Figure 34 provides an overview of the statistical distribution analysis step:

- **(a)** shows how the **histograms** get created, thanks to the **histograms.vi** in the statistics.lvlib, whose block diagram is shown in Figure 35b. Note that a histogram is created for each column in each dataset (ascending or descending) for both sensors. Each column represents a **feature**, which is a value of output current that drove the flow pump during the acquisition phase. Note that there is no **correct** no. of bins or bin width to be used, but many statisticians have suggested various formulas [12].
- **(b)** shows how the median value, lower and upper quartiles (Q1, Q3) and whiskers (U1, L1) are computed thanks to the **mediano.vi** and **whisky.vi** subVIs in the statistics.lvlib, whose block diagrams are shown in Figure 36b and 37b. The first subVI computes the median to divide the ordered dataset (a feature column) into two halves. The median is not included in either half. The lower quartile value is the median of the lower half of the data. The upper quartile value is the median of the upper half of the data.

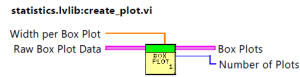


(a) histogram.vi SubVI Description

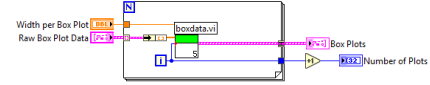


(b) histogram.vi SubVI Block Diagram

Figure 35: **histogram.vi** subVI used to create all the histograms for each feature column in a given dataset

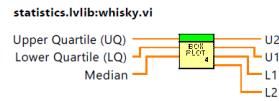


(a) create_plot.vi SubVI Description

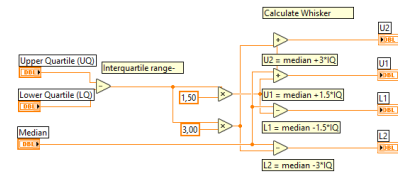


(b) create_plot.vi SubVI Block Diagram

Figure 36: **create_plot.vi** subVI used to create all the boxplots for each feature column in a given dataset

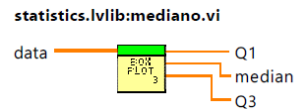


(a) whisky.vi SubVI Description

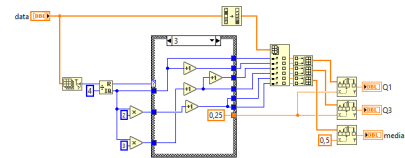


(b) whisky.vi SubVI Block Diagram

Figure 37: **whisky.vi** subVI used to compute the lower and upper whiskers for each feature column in a given dataset



(a) mediano.vi SubVI Description

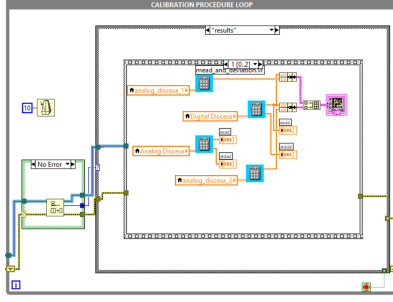


(b) mediano.vi SubVI Block Diagram

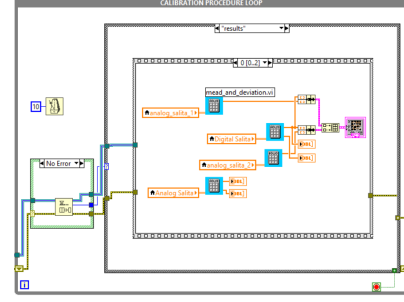
Figure 38: **mediano.vi** subVI used to compute the lower and upper quartiles and the median value for each feature column in a given dataset

Results: Expected Value and Standard Deviation

The next step is the evaluation of the expected values from the transducers datasets acquired during both ascending and descending phases. This is achieved in the **results** state in the *Calibration Procedure Loop*.



(a) Compute samples mean and standard deviations from samples acquired for analog sensor descending phase datasets.



(b) Compute samples mean and standard deviations from samples acquired for analog sensor ascending phase datasets.

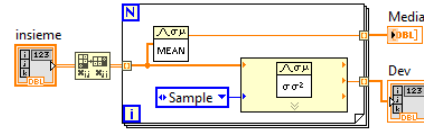
Figure 39: Calibration Procedure Loop: expected values and standard deviations evaluation phase.

Figure 39 provides an overview of the first results state:

- **(a)** This frame illustrates the updating process of the indicators shown on the front panel that plot ascending curve values for the Analog Sensor. The values computed are *expected values* values and *standard deviations* **point-by-point**. These values are computed by the subVI **mean_and_deviation.vi** in the CALIBRATION.lvlib, whose block diagram is shown in Figure 40b.
- **(b)** show the same process repeated for the descending phase.



(a) mean_and_deviation.vi SubVI Description



(b) mean_and_deviation.vi SubVI Block Diagram

Figure 40: **mean_and_deviation.vi** subVI used to compute *expected values* values and *standard deviations* in feature columns in each dataset

Standard Uncertainty Evaluation

By attaining to the **Guide to the expression of uncertainty in measurement** (GUM [1]) we compute the Type A Standard Uncertainty for both transducers.

Once a sufficient measurements of air flux and voltage are taken, both in ascending and descending phase, the following procedure is applied for both the analog and digital transducer:

1. The mean $\bar{\Phi}$ of the measurements:

$$\bar{\Phi} = \frac{1}{N} \sum_{i=1}^N \Phi_i$$

where N is the number of measurements and Φ_i are the individual air flux measurements.

2. The standard deviation σ_{Φ} of the measurements:

$$\sigma_{\Phi} = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (\Phi_i - \bar{\Phi})^2}$$

3. The **standard uncertainty** u_{Φ} , calculated as the standard deviation divided by the square root of the number of measurements N :

$$u_{\Phi} = \frac{\sigma_{\Phi}}{\sqrt{N}}$$

4. After calculating the standard uncertainty for each air flux and voltage measures, the maximum value of standard uncertainty in the upward direction U_{up} and the downward direction U_{down} is determined.
5. The maximum expanded uncertainty, obtained by multiplying the maximum uncertainty by the coverage factor $k = 2$:

$$U_{\Phi} = k * u_{max}$$

6. The maximum deviation between the uphill and downhill phase, Δ_{max} , is:

$$\Delta_{max} = \max\{|\Delta_i|\}$$

where Δ_i is given by:

$$\Delta = |\bar{\Phi}_{up_i} - \bar{\Phi}_{down_i}|$$

In words, Δ_i is the i -th absolute deviation between the ascending curve i -th expected value ($\bar{\Phi}_{up_i}$) and the descending curve i -th expected value ($\bar{\Phi}_{down_i}$), computed for each operating point tested generating a different current amplitude.

7. Finally, the combined uncertainty $U_{combined}$ was calculated as the square root of the sum of the

squares of the two maximum expanded uncertainties :

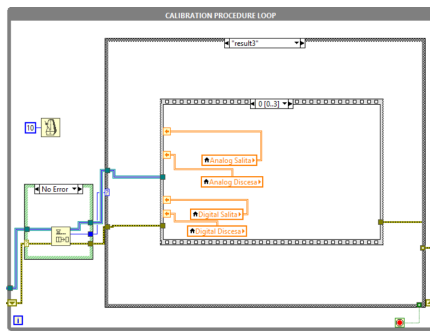
$$U_{combined} = \sqrt{U_{up}^2 + U_{down}^2}$$

8. The same procedure is applied for the analog sensor voltage measurements.

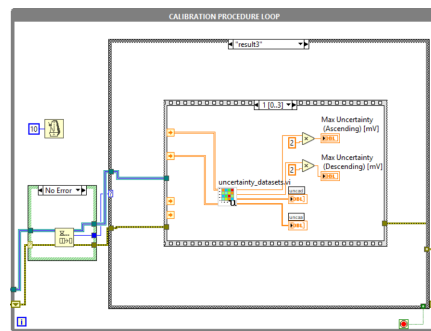
Results: Datasets Uncertainty

The program analyses both the ascending and descending phase datasets to compute the point-by-point uncertainty. Figure 41 provides an overview of the other results in this state:

- **(a)** shows how the sensor datasets are passed to the next frame via **sequence locals**.
- **(b)** illustrates how the **uncertainty_datasets.vi** included in the CALIBRATION.lvlib, and whose block diagram is shown in Figure 42b. This subVI computes the **point-by-point Type A Standard uncertainty** for both the ascending and descending curves. Further, the maximum values in both phases are returned on the output terminals of the subVI.



(a) Use the datasets in the next frames.



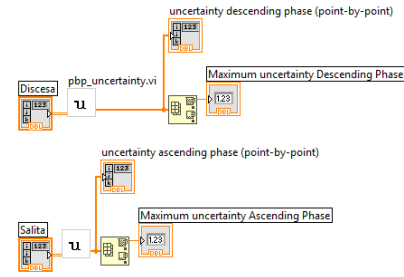
(b) Evaluation of point-by-point uncertainty.

Figure 41: Calibration Procedure Loop: Hysteresis evaluation phase.

CALIBRATION.lvlib:uncertainty_datasets.vi



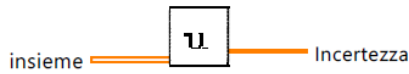
(a) uncertainty_datasets.vi SubVI Description



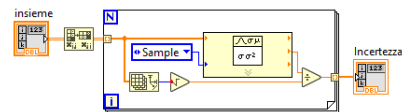
(b) uncertainty_datasets.vi SubVI Block Diagram

Figure 42: **uncertainty_datasets.vi** subVI used to compute the Type A standard uncertainty for each dataset feature.

CALIBRATION.lvlib:pbp_uncertainty.vi



(a) pbp_uncertainty.vi SubVI Description

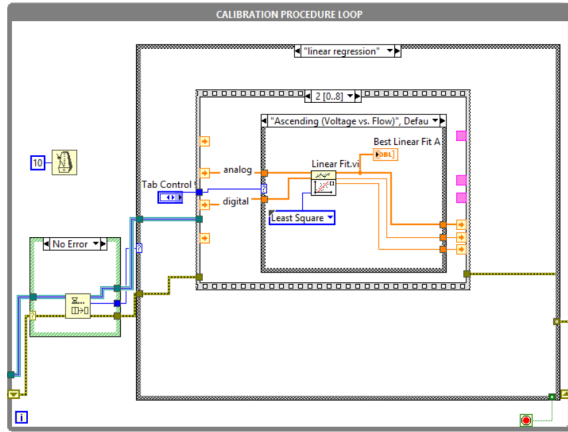


(b) pbp_uncertainty.vi SubVI Block Diagram

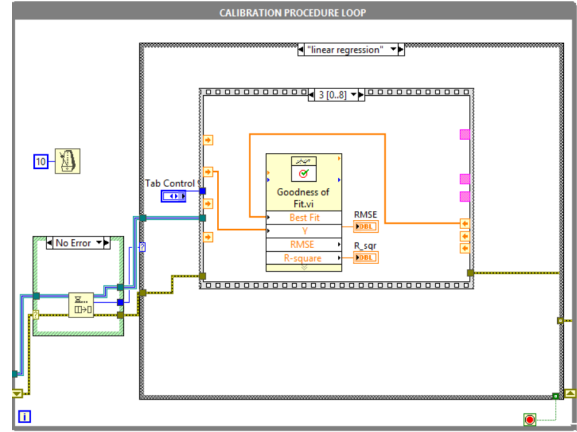
Figure 43: **pbp_uncertainty.vi** subVI used in the **uncertainty_datasets.vi** to compute the **point-by-point standard uncertainty**.

Linear Regression

After having evaluated the relevant statistical information for each phase dataset sample distribution, the next step is to find the best-fit estimators to the data curves derived from the sensors datasets. This will enable to prediction of the sensors output values starting from the input flow rate measured by the reference instrument in ascending and descending phase, respectively.



(a) Compute the **least-squares** linear estimator for the ascending phase.



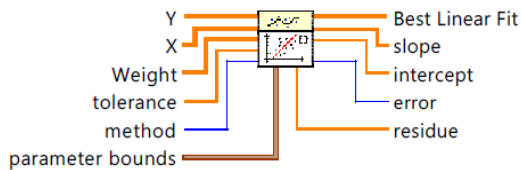
(b) Report the parameters that evaluate goodness of the approximation.

Figure 44: Calibration Procedure Loop: curve fitting phase.

Figure 44 provides an overview of the fitting state:

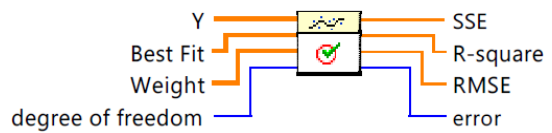
- (a) illustrate how the least squares estimator is searched for using the **Linear Fit.vi** [8].
- (b) shows how the coefficient of determination (R-square) and the Root Mean Square Error (RMSE) are computed by means of the **Goodness of Fit.vi**.

NI_AALPro.lvlib:Linear Fit.vi:5210001



(a) Linear Fit SubVI Description

NI_AALPro.lvlib:Goodness of Fit.vi

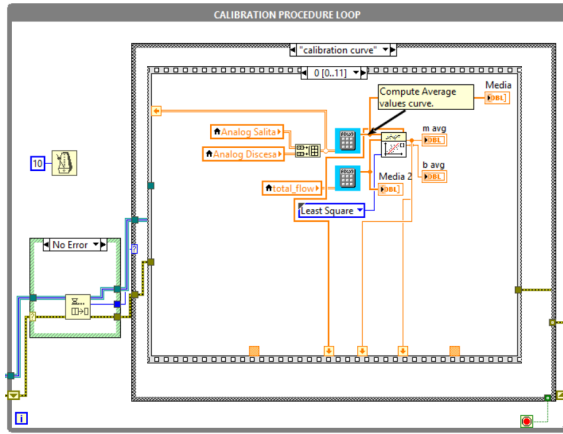


(b) Goodness of Fit SubVI Description

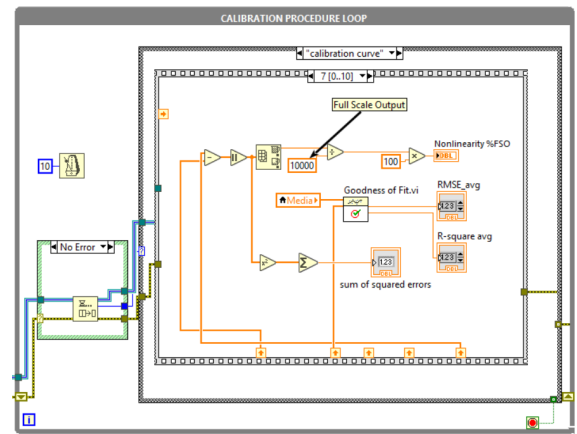
Figure 45: Curve Fitting subVIs used to compute the **least squares linear regression** estimators and the relative goodness parameters.

Calibration Curve

Finally, the program computes the calibration curve that will allow to predict the DUT (analog sensor) reading from the reference instrument reading.



(a) Search for Least Square Linear Estimator of the average curve.



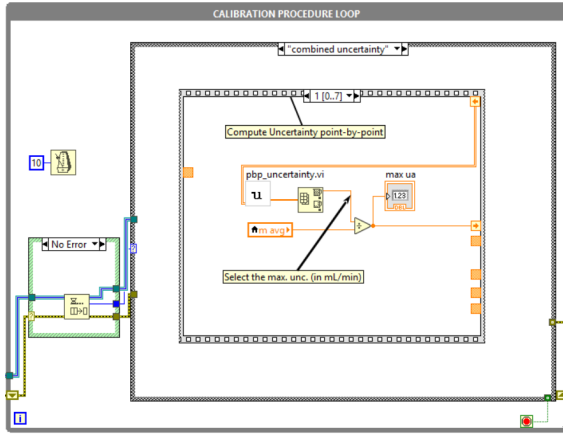
(b) Evaluate R-square, RMSE and linearity error in terms of % of FSO.

Figure 46: Calibration Procedure Loop: calibration curve linear regression.

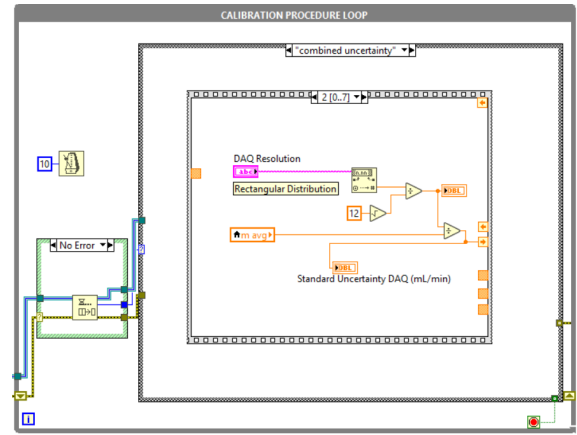
Figure 46 provides an overview of the calibration state:

- **(a)** shows the frame where, as in the previous state, the least squares linear regression estimator of the average curve is found by means of the *Linear Fit.vi*.
- **(b)** the just found linear regression estimator is used to evaluate the deviation of the average curve from the least squares line. Also, as previously seen in the linear regression state, the coefficient of determination and the RMSE value are also evaluated for this estimator.

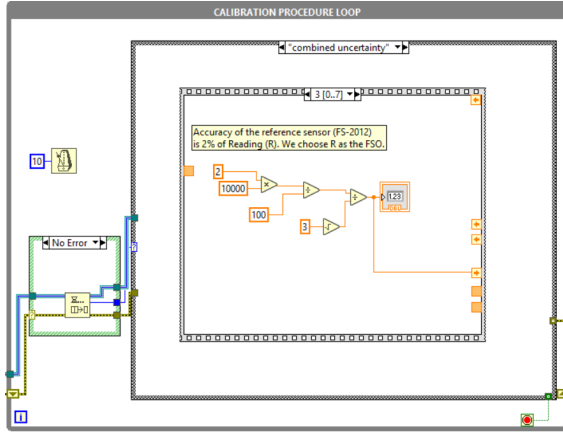
Combined Uncertainty



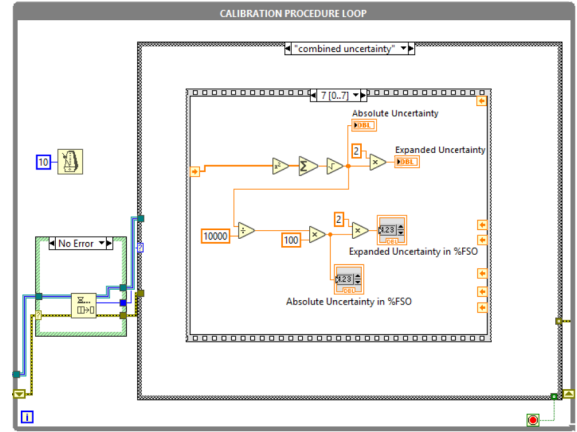
(a) Maximum Type A standard uncertainty from the dataset that include all datapoints.



(b) DAQ Resolution: Standard Uncertainty Evaluation (Type B, Rectangular Distribution).



(c) Reference Instrument Accuracy uncertainty contribution.



(d) Combined, Absolute and Expanded Uncertainty Evaluation.

Figure 47: Calibration Procedure Loop: measuring system combined uncertainty.

Figure 47 provides an overview of the measuring system combined uncertainty evaluation procedure:

- **(a)** shows the frame the uncertainty contribution due to the repeatability of the analog sensor measurements is evaluated.
- **(b)** shows the frame the uncertainty contribution due to the DAQ resolution is evaluated, by assuming a rectangular distribution.
- **(c)** illustrates how the uncertainty contribution for the digital sensor (reference instrument) is

computed. The reported accuracy is 2% of Reading. We choose to use the Full Scale Output as the value to assign to reading.

- **(d)** shows how the *absolute combined uncertainty* is evaluated, and expressed in terms of the percentage of the full scale output. The Expanded uncertainty is evaluated with a 95% confidence level, using an approximated coverage factor equal to 2.

Calibration Report

A Certificate of Calibration is a formal document that attests to the accuracy and performance of a measuring instrument or device. It serves as official proof that the instrument has been calibrated against traceable standards, ensuring its measurements are reliable and meet specified requirements. In the report **automatically generated by the main.vi**, the results section explains the procedure followed during the calibration process. The report also displays a table that contains the expected value and expanded uncertainty for each sensor, and for each operating point that has been tested, generating several air pump current amplitude inputs. An extensive uncertainty budget was developed, combining Type A uncertainty (from repeatability) with several Type B sources, such as the DAQ's resolution. All uncertainties were combined using Root Sum of Squares.

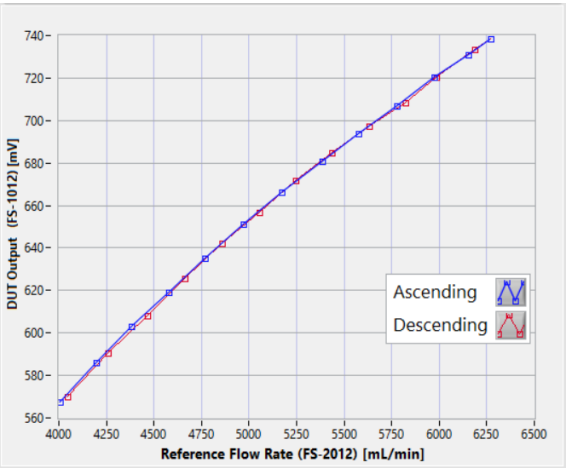
A dedicated LabVIEW Library, *Certification.lvlib*, and two subVIs were created for the purpose of reporting the results of the calibration procedure: *convert_table.vi* and *report.vi*. These subVI's icons are used within the calibration procedure loop to convert tables of numerical values to tables of string-valued elements, and to print the report programmatically, using the information that the user can see on the front panel of the Virtual Instrument.

Results

This chapter outlines a typical case scenario of the results that were obtained during test procedures.

Calibration Dataset Analysis

Ascending and Descending phases curves



(a)

Expected Values FS-1012 (mV)										
567,7	586,3	603,1	619,3	635,4	651	666,5	680,8	694,1	707,1	720,6

Standard Deviations FS-1012 (mV)										
2,2	10	2,1	1,7	1,9	1,6	1,8	1,7	1,8	1,7	1,8

Type A Standard Uncertainties FS-1012 (mV)										
0,2	0,7	0,1	0,1	0,1	0,1	0,1	0,1	0,1	0,1	0,1

(b)

Expected Values FS-1012 (mV)										
570,1	590,5	608,1	625,5	642,2	656,9	671,8	684,7	697,6	708,6	720,5

Standard Deviations FS-1012 (mV)										
3,5	2,8	1,9	1,8	1,7	2,2	1,7	1,7	1,6	1,7	2,2

Type A Standard Uncertainties FS-1012 (mV)										
0,2	0,2	0,1	0,1	0,1	0,2	0,1	0,1	0,1	0,1	0,2

(c)

Figure 48: FS-1012 (a) ascending and descending curves, with statistical parameters during (b) ascending phase (c) descending phase

When combining the curves from both sensors (in both ascending and descending phases), a direct relationship emerges between the outputs of the two instruments. When we manage to generate an air flow whose temperature is approximately constant, both instruments do not change their behaviour significantly. But if temperature varies, than it has been experienced that the ascending and descending phases measurements are affected by this disturbance and, hence, produce a different output in correspondence of the same input flow rate, as it can be seen in the following in Figure 49:

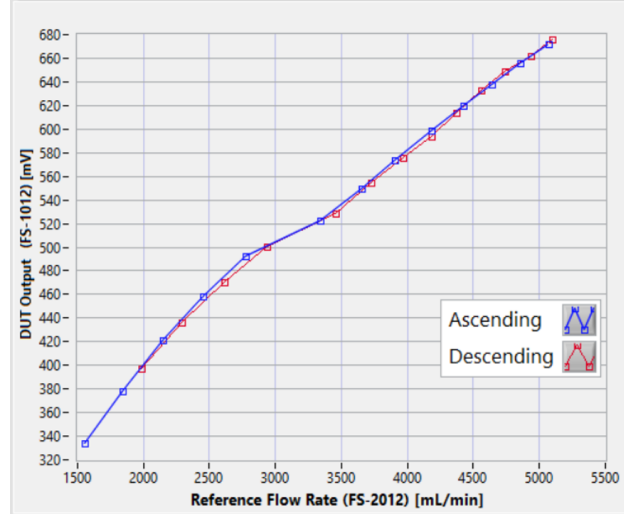


Figure 49: Ascending and Descending phases output when the temperature of the flowing gas is not held constant during the calibration cycle.

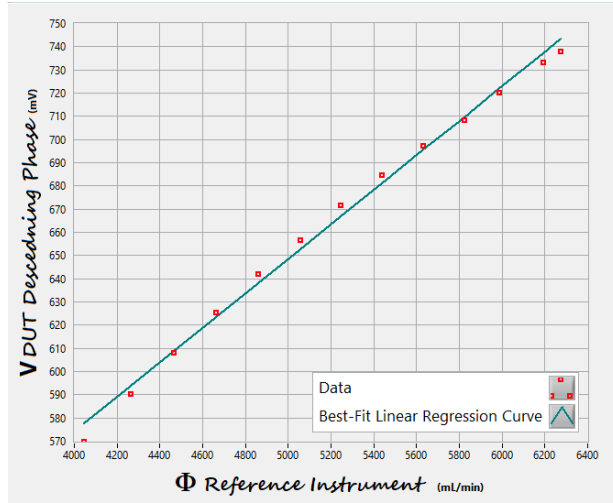
Ascending and Descending phases: Linear Regression

For both the ascending and descending phases curves a linear regression estimator was computed yielding approximately the same results.

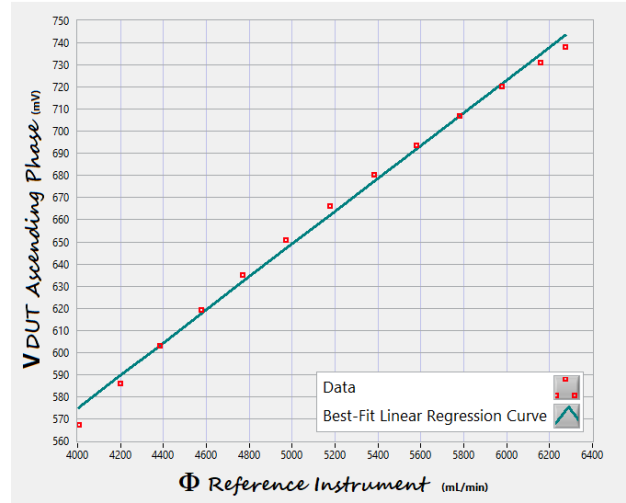
The linear regression model in both cases is :

$$V = (m \cdot \Phi + b)mV \quad (VI.1)$$

where m and b are the slope and intercept of the linear regression curve, and Φ is the input flow rate. The slope and intercept of linear regression line for the ascending phase are $m_{asc} = 0.07 \frac{mV}{mL/min}$, $b_{asc} = 277.5mV$. The slope and intercept of linear regression line for the descending phase are $m_{asc} = 0.07 \frac{mV}{mL/min}$, $b_{asc} = 276.9mV$. The linearity error measured in both cases amounts to 1% of FSO. The



(a) Ascending Phase Curve Least Squares Linear Regression

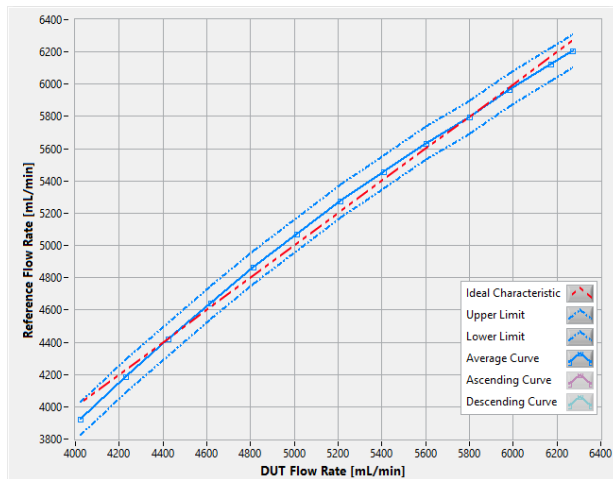


(b) Descending Phase Curve Least Squares Linear Regression

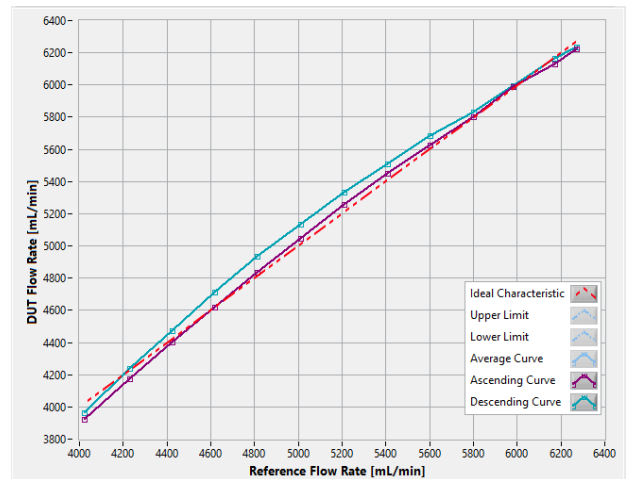
Figure 50: Best-Fit Estimators: FS-1012 (a), FS-2012 (b)

coefficient of determination, R^2 , is calculated to be 0.995 in both cases, respectively. This high value indicates an excellent fit of the linear regression curve to the measured data, suggesting that the model reliably represents the relationship between inputs and outputs.

Calibration Curve and Hysteresis Effect



(a) Calibration Curve.



(b) Hysteresis Effect.

Figure 51: Calibration Results

The calibration curve is obtained from the linear regression on the dataset comprising all measurements acquired during the calibration cycle (both ascending and descending phase).

Also in this case, the linear regression model is :

$$V = (m \cdot \Phi + b)mV \quad (VI.2)$$

where $m = 0.07 \frac{mV}{mL/min}$, $b = 277.2mV$. The coefficient of determination, R^2 , is calculated to be 1. Again, this value indicates an excellent fit of the calibration curve to the measured data. At this point, it is possible to compute the maximum deviation between the ascending and descending phases curves and compare the maximum deviation with the ascending and descending phase combined standard uncertainty, evaluated as the Root Sum of Squares (RSS) value, taking into account the maximum uncertainty in each phase. The maximum deviation is found to be equal to $\Delta = 100.2 \frac{mL}{min}$, while the combined uncertainty is equal to $u_{comb} = 20.5 \frac{mL}{min}$. This suggests that the DUT output is affected by hysteresis. Also, the linearity error, computed as the maximum deviation of the average curve from the ideal characteristic, is reported as 1%FSO. On the calibration curve plot, also upper and lower limits appear, which are obtained as follows.

Measuring System Combined Uncertainty

As presented in the *Calibration* chapter of this manuscript, the *main.vi* program is capable of producing a *calibration report*. In this context, the final result from the calibration procedure is produced: the *Combined Standard Uncertainty* of the Measuring System. This value arises from the *Uncertainty Budget*, which considers all the relevant *Sources of Uncertainty* in the Measuring System. From the known distribution of each source, a *standard uncertainty* is evaluated.

<i>Source Of Uncertainty</i>	<i>Estimate</i>	<i>Distribution</i>	<i>Uncertainty Contribution [mL/min]</i>
<i>DUT Standard Error of the mean</i>	1.5 mV	Type A, Normal	20.5
<i>DAQ Resolution</i>	4.88 mV	Type B, Rectangular	19
<i>Reference Instrument Accuracy (% of reading)</i>	2 % of Reading	Type B, Rectangular	115.5

Figure 52: Uncertainty Budget, with all the relevant sources of uncertainty in the measuring system.

Among the sources of uncertainty that were identified in the measuring systems we account for the **DAQ Resolution**, the **DUT Standard Error of the mean**, and the **Reference Instrument Accuracy** reported in the device datasheet. We also include the uncertainty contribution due to hysteresis effect and non linearity. This brings us to the final results:

Measuring System Combined Standard Uncertainty : $134.9 \frac{mL}{min}$

Expanded Combined Uncertainty ($k = 2$): $269.9 \frac{mL}{min}$

Absolute Uncertainty: 1.35% FSO

Expanded Uncertainty ($k = 2$): 2.7% FSO

Conclusions

Project Workflow

This project's aim was the development of an automated measurement system for characterizing an air flow transducer, specifically the **Renesas FS-1012-1100-NG** thermal mass analog flow sensor. The calibration has been carried out versus a calibrated digital air flow sensor, precisely the **Renesas FS-2012-1100-NG** thermal mass digital flow sensor. The *project workflow* involved the following steps:

- the signal acquisition from the FS1012 transducer signal conditioning circuit;
- the signal acquisition from the FS2012 transducer signal conditioning circuit;
- Calibration of the FS1012 transducers versus the calibrated FS2012 transducer;
- Control of the PP2 air pump by means a programmable benchtop laboratory power supply;
- Display of the air flow from the FS1012 transducer and the FS2012 transducer in real-time.

Nevertheless, these tasks were addressed only after having considered the physical quantities that have to be dealt with in the measuring system, such as the output voltage of the **Device Under Test (DUT)**. This quantity needed careful consideration before it could be used to represent the variation of the air flow rate generated by the air pump. Indeed, when measured by a multimeter, the DUT's **Full Scale Output (FSO)**– at the maximum air flow rate allowed by the potential risk of overheating the air pump– quantifies to a few millivolts at most. This value depends on the difference between the sensor's thermopiles voltages (TP1 & TP2). Thus, a **conditioning circuit** was designed and built on a soldering

board with the aim of amplifying the two voltage sources separately and then computing their difference. This operation provided a feasible voltage magnitude to be acquired by a data acquisition board.

To carry out the tasks required to calibrate the DUT, a set of software applications was developed within the **LabVIEW** development environment, with the *main.vi* being the primary *Virtual Instrument* to be used for the calibration procedure.

The set of device connection statuses (establish, write command, read data, etc.) that had to be modelled for each measuring instrument interface led to the choice of the **Finite State Machine (FSM)** software paradigm, together with the **Producer-Consumer** design pattern. This choice, revealed later to be a significant advantage, since it enabled to manage each device connection separately and in parallel with the others. Moreover, it significantly decreased the application response time for every user request via the *graphical user interface (GUI)* developed. The latter, also known as the VI *front panel*, has been designed to guide the user through the calibration procedure, exposing only the relevant controls that should be used for each step in the process, hiding other parameters in dedicated subfolders. The description of the GUI developed for the *main.vi* is given in the User's Manual that has been written for this project's software application.

For each user interaction (*event*), with the GUI an application behaviour was defined to manage the user request, represented by the event source ("Read button pushed", "Connect to Arduino", etc.). The **Queue** software structure was used in order to define an ordered sequence of operations to handle each user request.

The *main.vi* is thought to be used by the user to control the air pump- via the Power Supply Unit (PSU) controlled remotely- and the two air flow rate transducers: the DUT and the reference digital transducer. Three subVIs, each implementing a FSM, were used to handle the connection with the remote PSU (**GPIB connection**), the connection with DAQ board (**DAQmx Tasks**), and the Arduino board (**Serial connection**), respectively. Both the *Agilent_GPIB_FSM.vi* and *Arduino_Serial_FSM.vi* use the **VISA (Virtual Instrument Software Architecture)** subpalette. This subpalette provides a unified, device-independent application programming interface for communicating with various types of instruments and devices, and it is used to send commands and retrieve data from GPIB and Serial device interfaces. The *daq_FSM.vi*, instead, uses the set of functions from the **NI DAQmx** subpalette. This subpalette, allows to control the DAQ various types of measurements or signal generations, including the

analog input reading operation, needed to capture the DUT's output voltage.

When executed, the main.vi uses the **Start an Asynchronous Call** function to call each of these programs during the initialization phase of the application. This means that the application does not wait for the subroutine to end, but instead continues to execute in parallel to the processes it calls. This is a further improvement in terms of program response time, and it has been possible thanks to the Queue Operations used.

Finally, a set of subVIs was developed to address the *data analysis* phase of the calibration procedure. These subVIs were collected under different *LabVIEW Libraries* to differentiate between their purpose. Another library, *Calibration.lvlib*, includes all the subVIs that are used by the main.vi to analyse the calibration datasets extensively. These subVI are capable of computing the expected value and standard deviation of each feature column and evaluating the **standard uncertainty** associated with the results found for the ascending and descending phase datasets collected during the acquisition procedure. The last step involved the estimation of a linear regressor that could be used to predict a flow rate measure comparable with that of the reference sensor, starting from the raw voltage output of the DUT. Also, depending on the ascending or descending phase, a different conversion equation can be used.

Concluding Remarks

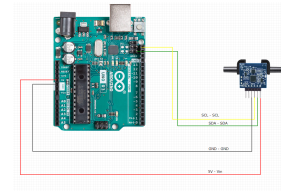
The datasets collected during the testing phase of this project were analysed with these subVIs. The comparison between the two sensors' output curves, during the ascending and descending phases, revealed that **there is a relationship between the two transducers' outputs**. This relationship has been estimated by linear regression, which in both the ascending and descending phase dataset resulted in an acceptable approximation. To quantify the measurement system overall uncertainty, and to provide a report of the expected values and standard uncertainties for each operating point, a **calibration report** was generated by the *report.vi* subVI, within the *Calibration Procedure Loop* in the *main.vi*.

Arduino Script

Circuit

Pin FS-2012	Wire Colour	Pin Arduino
Vin	Red	5V
GND	Black	GND
SDA	Green	SDA
SCL	Yellow	SCL

(a) Arduino UNO - Reference Digital Sensor Pins Assignment



(b) Arduino UNO - Reference Digital Wiring

Figure 53: Overview of the Arduino UNO R3 - Digital Transducer Connection.

This Appendix details the Arduino script, found in the project files, responsible for acquiring digital sensor measurements after converting the transducer's output via its datasheet-specified function.

Library Inclusions

The sketch utilizes essential libraries: **Wire.h** for fundamental I²C communication, and **Arduino.h** for core Arduino functionalities. **I2Cdev.h** is included but not explicitly used in the current version.

Pin Definitions and Parameters

Hardware connections are defined by standard I²C pins on Arduino Uno: **SDA (A4)** for data and **SCL (A5)** for clock. The fixed I²C slave address for the FS2012 sensor is **FS2012_ADDRESS (0x07)**.

Global Variables

Key global variables include **data** (16-bit raw sensor output), **flow** (converted air flow value), and **delayMs** (delay between readings, initially 500 ms for 2 Hz sampling).

Sensor Reading Function: *digital_flow_sensor()*

This function encapsulates the I²C protocol for reading the FS2012 sensor. It initiates a transmission to **FS2012_ADDRESS**, requests reading from register 1, and then retrieves two bytes. These bytes are combined (**a** << 8 | **b**) into a 16-bit **data** value, which is subsequently converted to L/min by dividing by 1000.0.

Initialization Function: *setup()*

The **setup()** function performs initializations upon Arduino startup. It sets up serial communication at 9600 baud using **Serial.begin(9600)**, initiates the I²C bus with **Wire.begin()**, and includes a 500 ms delay for stabilization.

Continuous Execution Function: *loop()*

The **loop()** function runs continuously, managing the primary operational flow. It features serial command control to modify the sampling rate (e.g., via **SF:<frequency>**), calls **digital_flow_sensor()** to update the **flow** value, incorporates a **delayMs** pause, and prints the measured **flow** value to the serial monitor.

Serial Event Handler Function: *serialEvent(String command)*

This function, invoked when serial data is available, parses and acts upon incoming commands. It trims the **command** string, checks for the "SF:" prefix, extracts the frequency value, converts it to a float, and, if positive, calculates and updates **delayMs** (e.g., **delayMs = (unsigned long)(1000.0 / newFreq);**). This mechanism enables dynamic adjustment of the sensor's sampling rate via serial commands.

Bibliography

- [1] JCGM 100:2008. *Evaluation of Measurement Data — Guide to the Expression of Uncertainty in Measurement*. 2008. URL: https://www.bipm.org/documents/20126/2071204/JCGM_100_2008_E.pdf.
- [2] FS-1012. *FS-1012-1100-NG Datasheet*. URL: <https://www.renesas.com/en/products/fs1012>.
- [3] FS-2012. *FS-2012-1100-NG Datasheet*. URL: <https://www.renesas.com/en/products/fs2012>.
- [4] *Agilent e3634A Datasheet*. URL: <https://www.mouser.com/pdfdocs/2sgE3634-90010.pdf>.
- [5] *Arduino UNO R3 Datasheet*. URL: <https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf>.
- [6] Thomas Bress. *Effective LabVIEW Programming, Chapter 4, Classical State Machines*. URL: https://www.researchgate.net/publication/331408353_Effective_LabVIEW_Programming.
- [7] Afshin Ghajar and Yunus Cengel. *Heat and Mass Transfer - Fundamentals and Applications, 6th Edition*, McGraw-Hill Education, New York, NY, 2020. Apr. 2021.
- [8] LabVIEW Programming Reference Manual. *Fitting*. URL: <https://www.ni.com/docs/en-US/bundle/labview-api-ref/page/vi-lib/analysis/6fits-llb/linear-fit-vi.html>.
- [9] A.S. Morris and R. Langari. *Measurement and Instrumentation: Theory and Application*. Academic Press, 2020. ISBN: 9780128171424. URL: <https://books.google.it/books?id=z7vbDwAAQBAJ>.
- [10] *NI USB-6008 Datasheet*. URL: <https://download.ni.com/support/manuals/375295c.pdf>.

- [11] Lisa K. Wells and Jeffrey Travis. *LabVIEW for Everyone*. URL: https://books.google.com/books/about/LabVIEW_for_Everyone.html?id=cGDGAQAACAAJ.
- [12] Wikipedia. *Histograms, Number of Bins and Bin Width Rules*. URL: https://en.wikipedia.org/wiki/Histogram#Number_of_bins_and_width.